The Systems Behind Modern AI

Why and how does hardware designed for 3D games power modern AI

Sam Ly

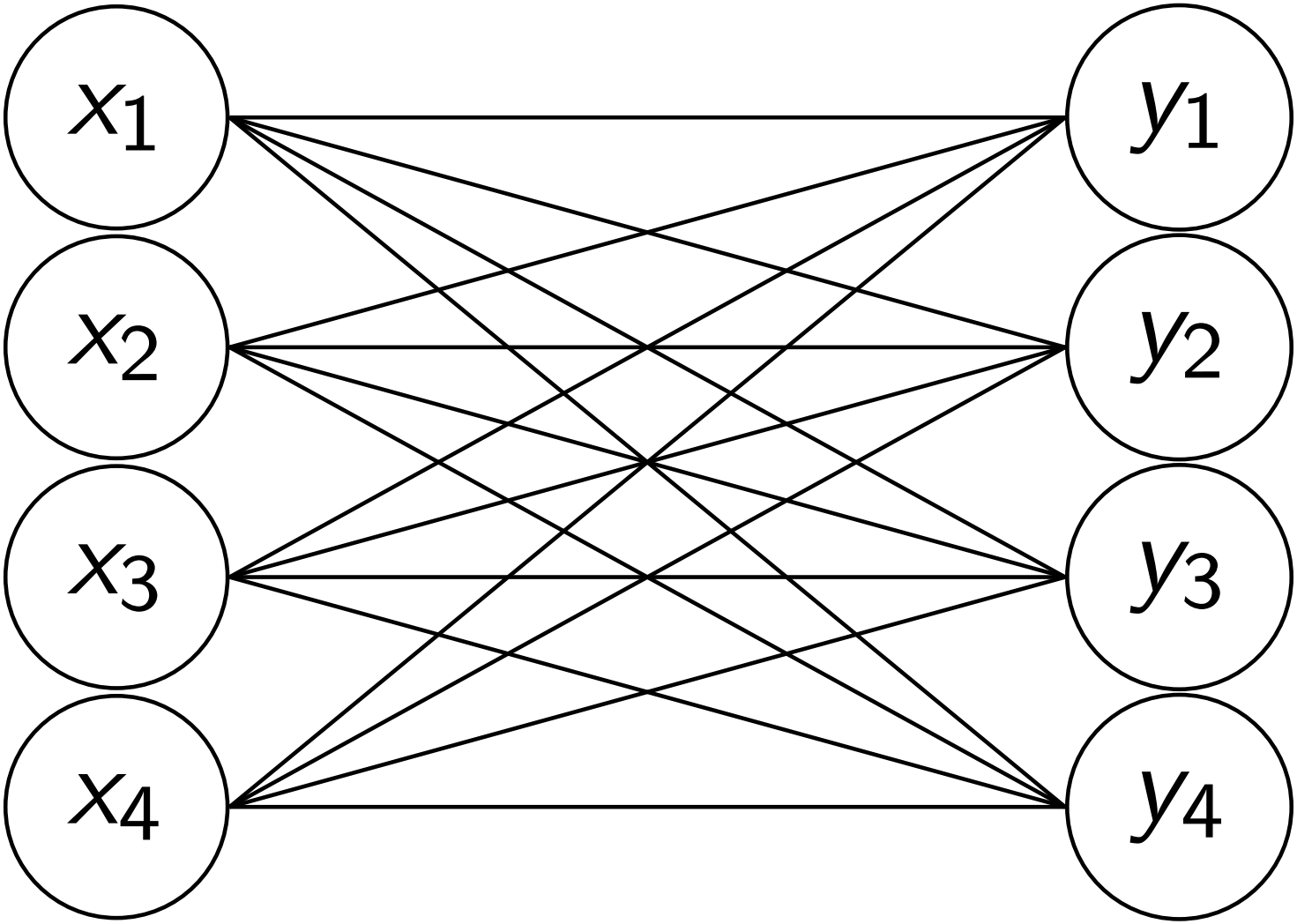
Cal Poly Pomona

Spring 2026

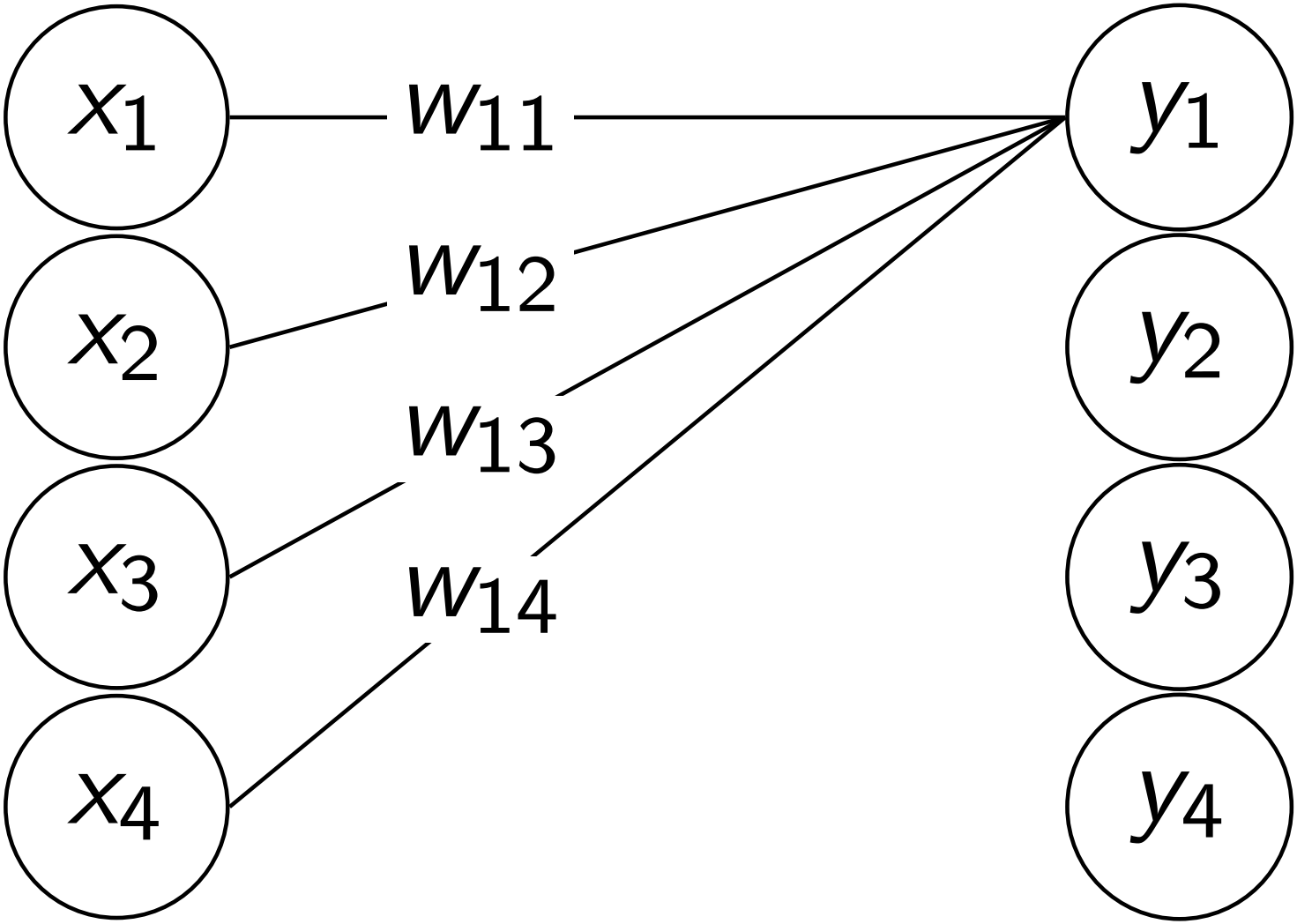
Table of Contents

1. Neural Networks and the Matrix Multiplication
2. Why GPUs? What's wrong with CPUs?
3. From graphics pipeline to AI accelerator
4. Optimizing **Ge**neral **M**atrix **M**ultiplication (GEMM) on CUDA
5. Future discussions
6. Conclusion and References

Basic Neural Net (MLP/FFN)

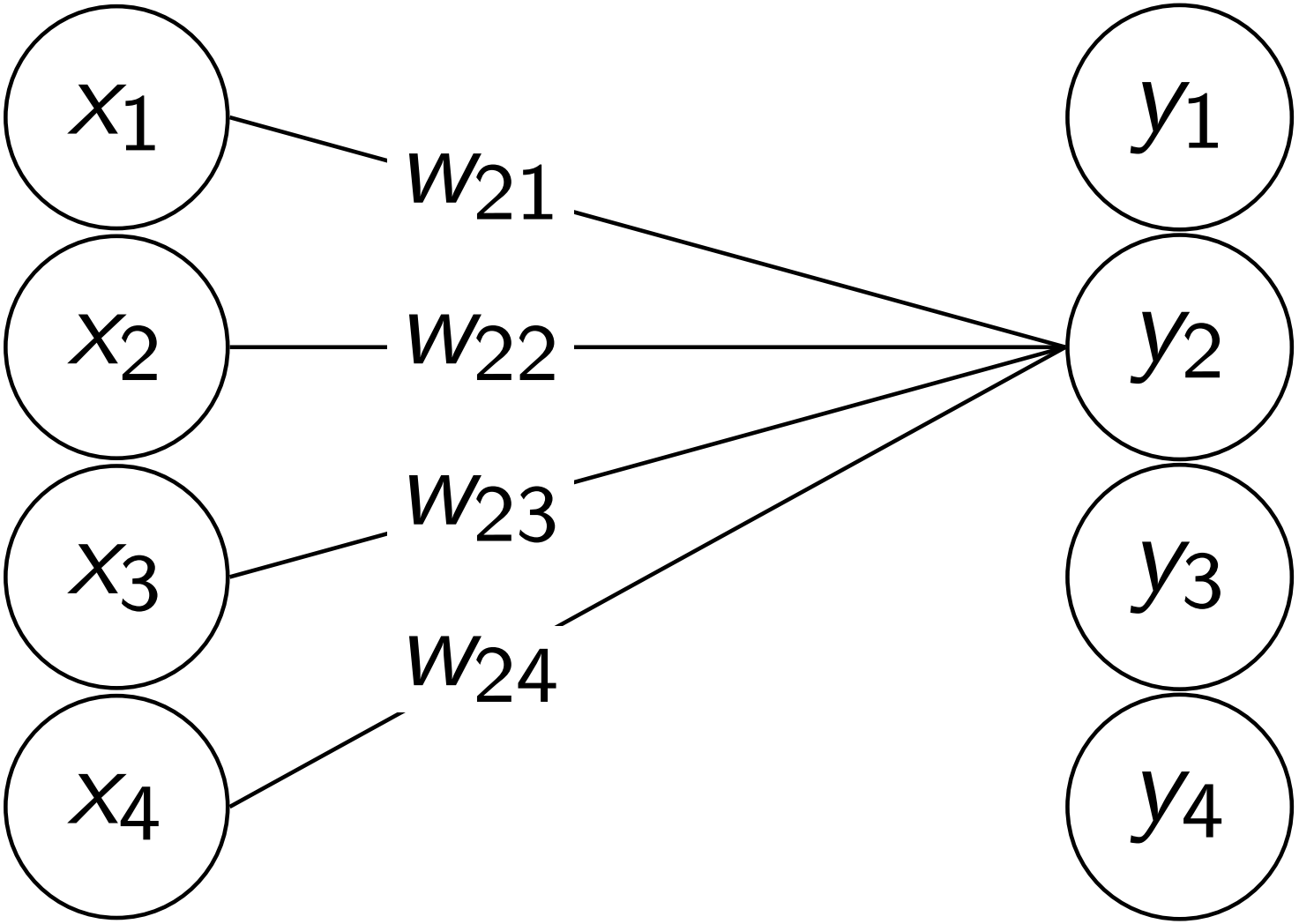


Basic Neural Net (MLP/FFN)



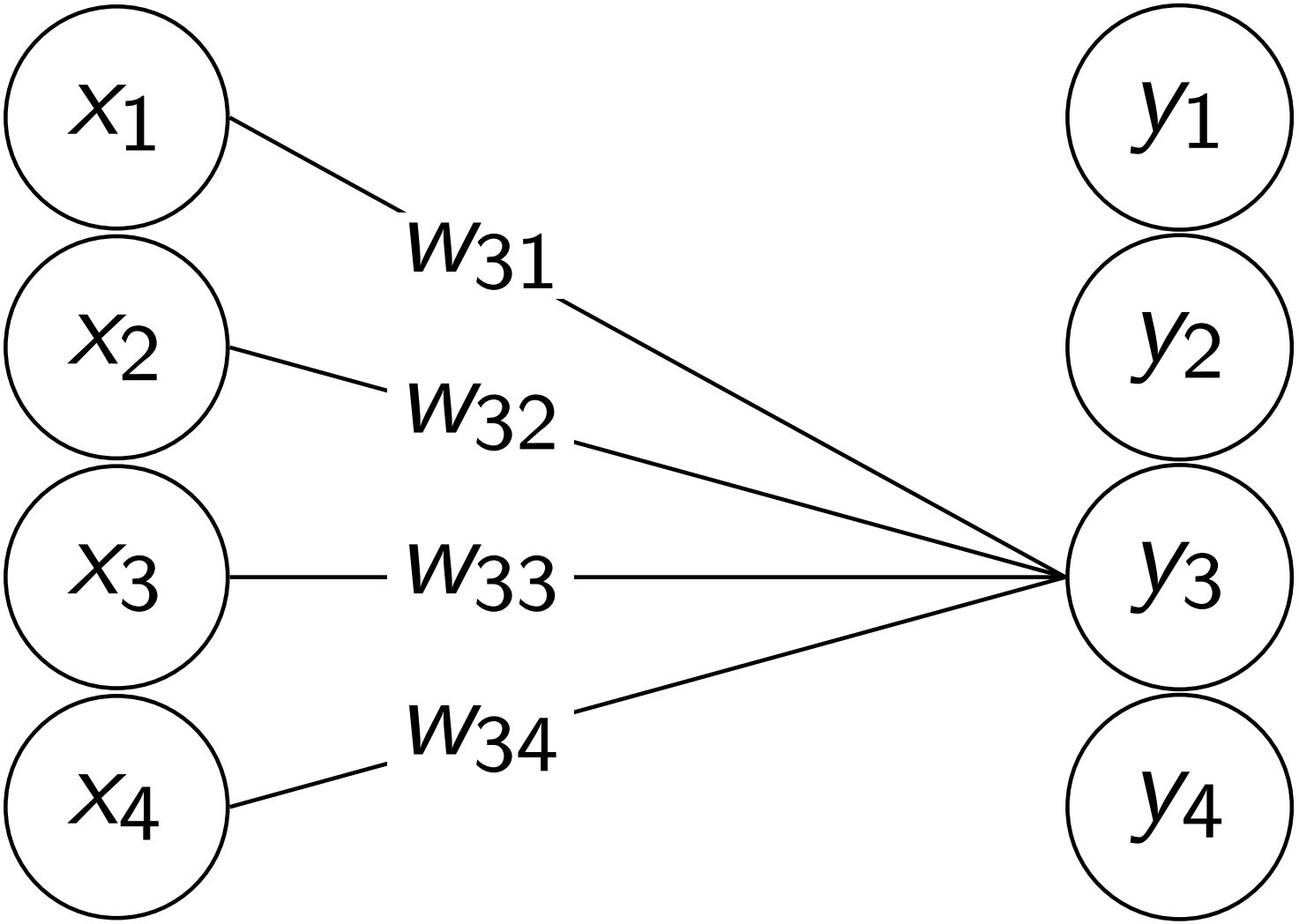
$$y_1 = x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14}$$

Basic Neural Net (MLP/FFN)



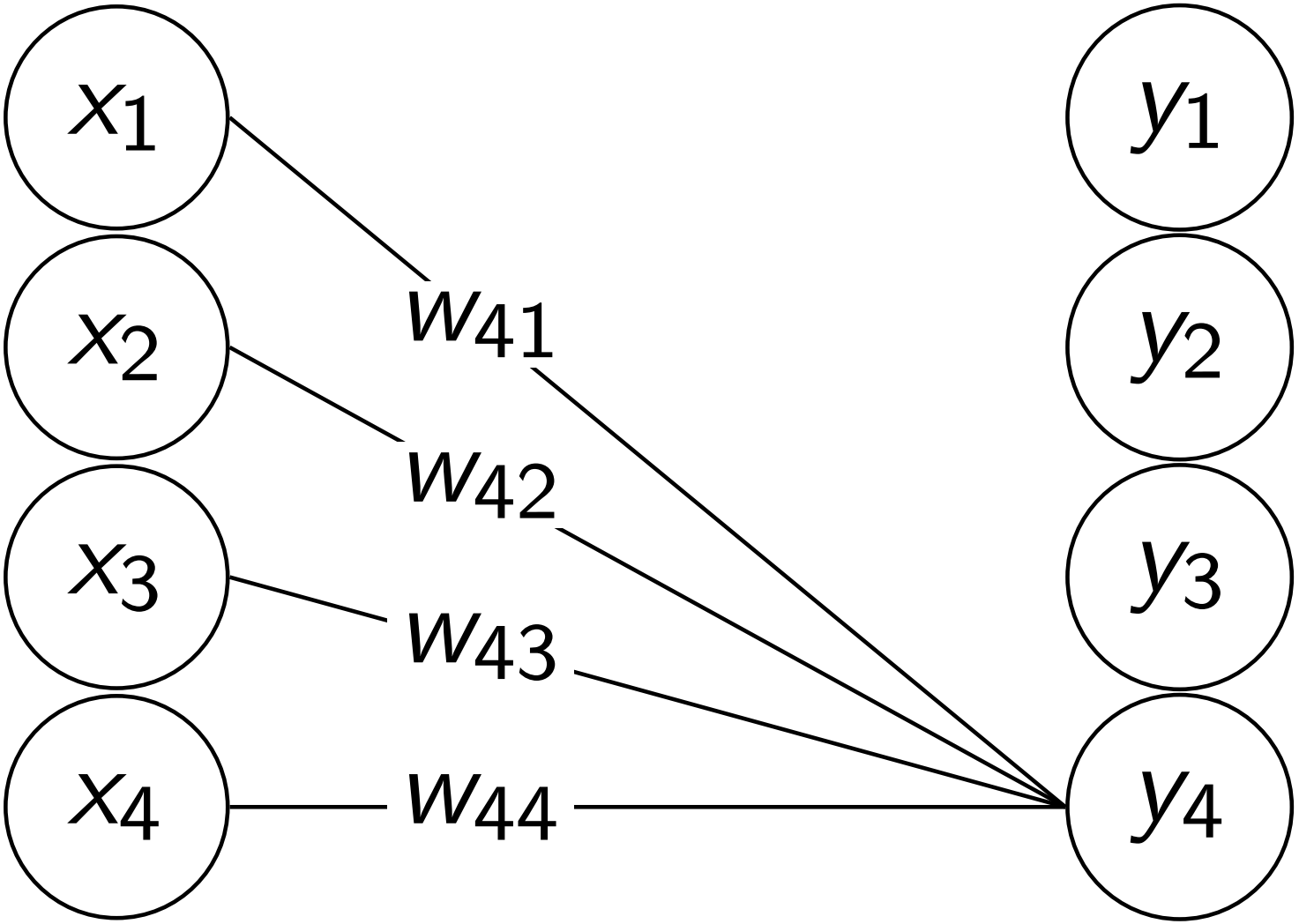
$$y_2 = x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24}$$

Basic Neural Net (MLP/FFN)



$$y_3 = x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34}$$

Basic Neural Net (MLP/FFN)



$$y_4 = x_1 w_{41} + x_2 w_{42} + x_3 w_{43} + x_4 w_{44}$$

Linear Algebra Interpretation

Here, notice that for each y_i, we are taking a weighted sum of the x elements. That is,

$$y_i = x_1 w_{i1} + x_2 w_{i2} + x_3 w_{i3} + x_4 w_{i4}.$$

Arranging the y_i into a vector, we get

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \\ w_{31} & w_{32} & w_{33} & w_{34} \\ w_{41} & w_{42} & w_{43} & w_{44} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}.$$

Linear Algebra Interpretation

We can generalize the number of x's and the number of y's.

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

We say, n is the *input dimension* and m is the *output dimension*.

Computing with multiple x vectors.

Suppose we have multiple x vectors, for which we want to pass through our neural network. We can stack the multiple x vectors in the **batch**, which we compute with a matrix multiplication.

$$\begin{bmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(B)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ y_m^{(1)} & y_m^{(2)} & \cdots & y_m^{(B)} \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(B)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ x_n^{(1)} & x_n^{(2)} & \cdots & x_n^{(B)} \end{bmatrix}.$$

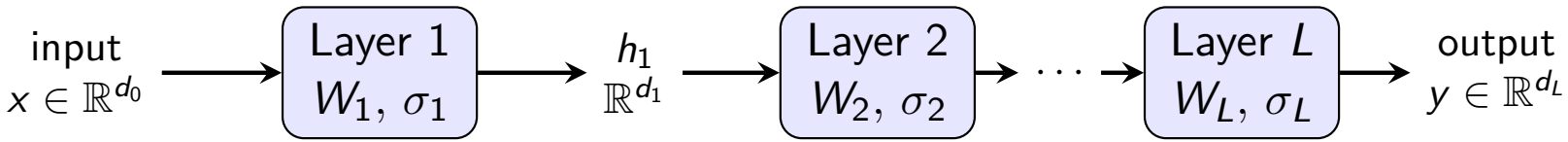
We write this succinctly as $Y = WX$, where $Y \in \mathbb{R}^{m \times B}$, $X \in \mathbb{R}^{n \times B}$, and $W \in \mathbb{R}^{m \times n}$.

Yippee!

We have successfully mapped the batched computation of a **linear layer** a matrix multiplication!

Composing Layers

We can view W as a function that takes in an n-dimensional vector and maps it to an m-dimensional vector. $W : \mathbb{R}^n \to \mathbb{R}^m$. We create deep neural networks by *composing* these transformations, along with **nonlinearities** σ_i.

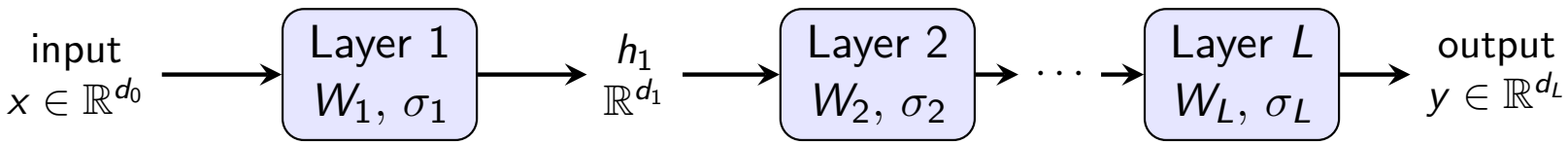


$$h_0 = x, \qquad h_\ell = \sigma_\ell(W_\ell h_{\ell-1}), \qquad y = h_L$$

Remark

Notice that this figure doesn't show the *batch* dimension. However, if you work through the dimension mappings, the *batch* dimension B will be unaffected by this composition.

Limitations for Neural Network Architecture



$$h_0 = x, \qquad h_\ell = \sigma_\ell(W_\ell h_{\ell-1}), \qquad y = h_L$$

Architectures such as **Transformers** and **CNNs** have connectivity patterns more complex than a simple composition of layers. Nevertheless, linear transformations remain a fundamental computational primitive in neural network architectures.

A fundamental building block

A linear transformation may seem like a narrow operation to study, but modern neural networks perform it repeatedly and at enormous scale. Its efficiency therefore has an outsized effect on training and inference performance. The rest of this presentation explores the hardware and techniques that make these operations efficient.

Takeaways

1. A linear layer applied to a batch of inputs can be expressed as a single matrix multiplication, $Y = WX$.
2. Deep networks compose these linear transformations with nonlinearities. More complex architectures change how layers are connected, but linear transformations remain a fundamental building block.
3. **Matrix multiplication therefore captures a large and reusable part of neural network computation.**

Why is this representation useful?

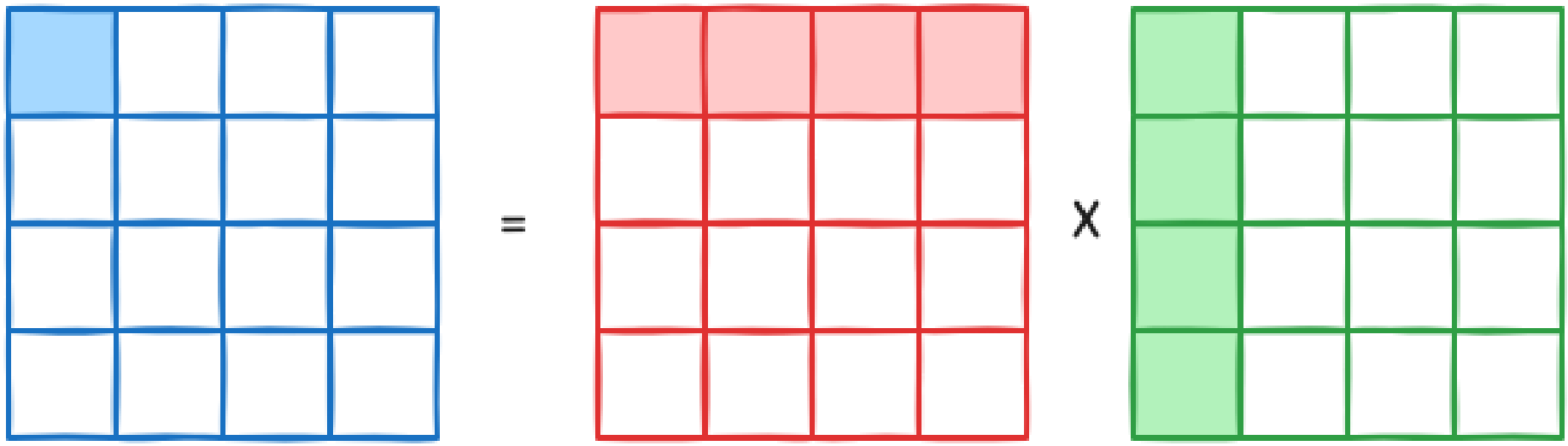
It separates what the network computes from how the hardware executes it. Instead of optimizing every architecture from scratch, we can focus on one common computational primitive. Next, we will examine how GPUs execute matrix multiplications efficiently.

Table of Contents

1. Neural Networks and the Matrix Multiplication
2. Why GPUs? What's wrong with CPUs?
3. From graphics pipeline to AI accelerator
4. Optimizing **Ge**neral **M**atrix **M**ultiplication (GEMM) on CUDA
5. Future discussions
6. Conclusion and References

Computing a Matrix Multiplication

Let A and B be two 4×4 matrices. The matrix multiple $C = AB$ results in another 4×4 matrix.



The operations are very simple, but the number of operations grow on the order of $\mathcal{O}(n^3)$, where n is the dimensions of the matrices.

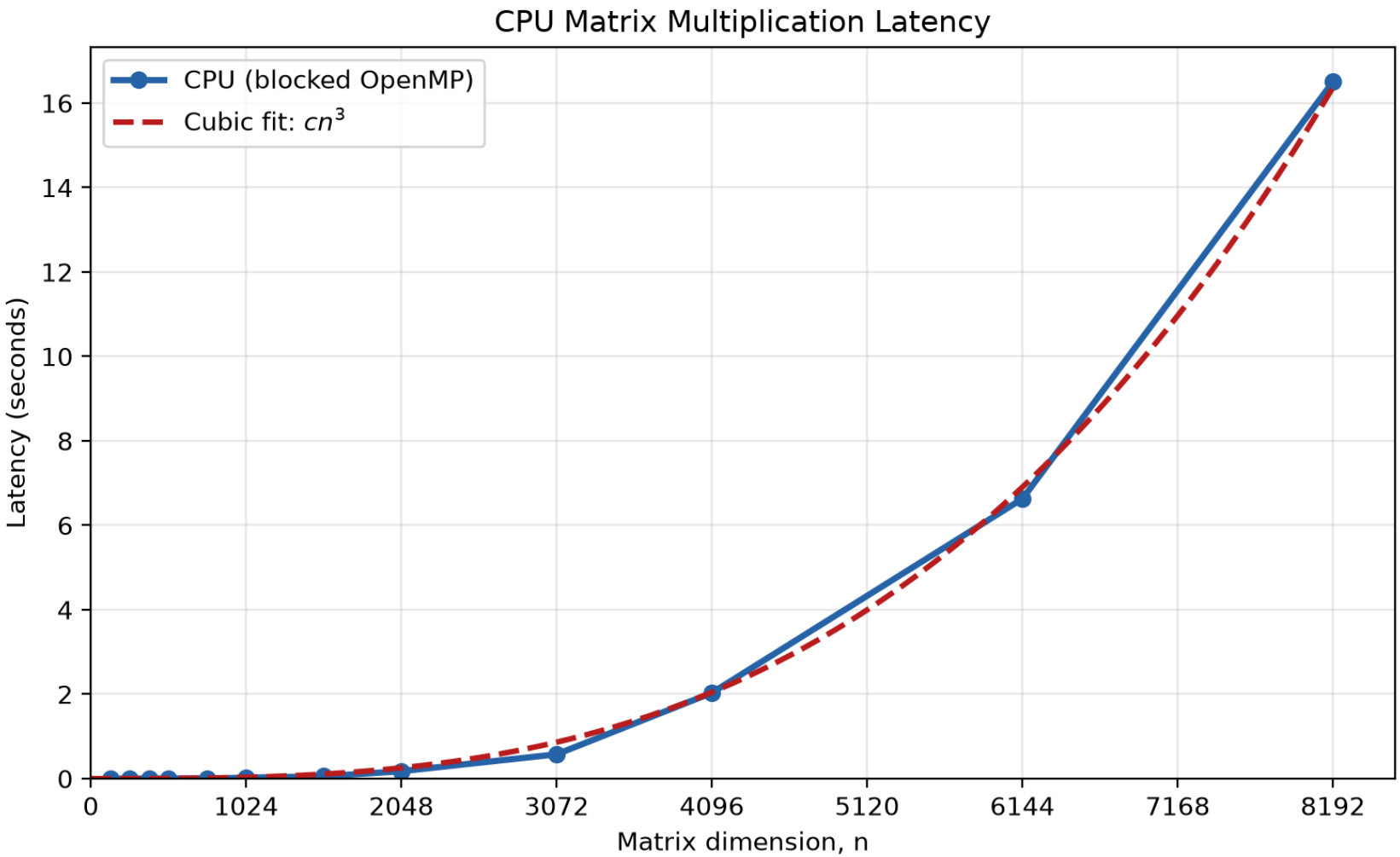
Remark

Linear algebra and matrix multiplications show up everywhere in scientific computing, so computing them efficiently has always been in high demand.

Runtime of a matrix multiplication on a CPU

```
for (std::size_t i = 0; i < n; ++i) {
  for (std::size_t j = 0; j < n; ++j) {
    double sum = 0.0;
    for (std::size_t k = 0; k < n; ++k) {
      sum += a[i*n+k] * b[k*n+j];
    }
    c[i*n+j] = sum;
  }
}
```

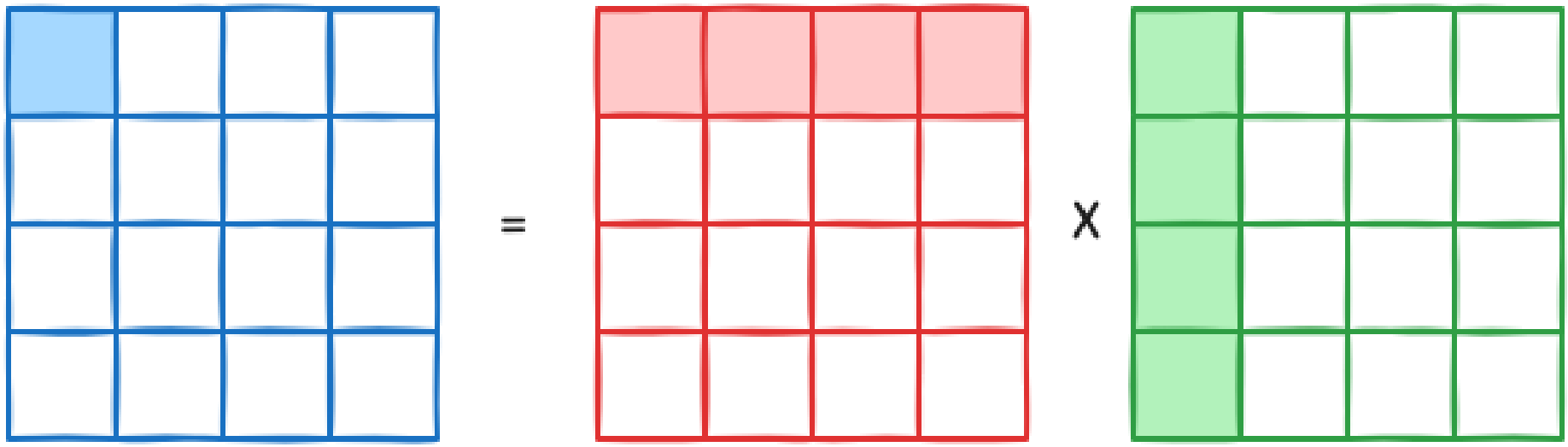
Runtime of a matrix multiplication on a CPU



Parallelism

GPU's came about due to the increase in computational power needed to render 3D games in real time. This requires *millions* of FLOP's per second. CPU's could not keep up. Rendering graphics has a nice property that we can take advantage of: **parallelism**.
Each pixel on the screen doesn't really care any of the other pixels. Furthermore, the computation that is required for each pixel isn't that complex either. The difficulty in rendering graphics is the volume of computations.

Matrix Multiplication is Parallel

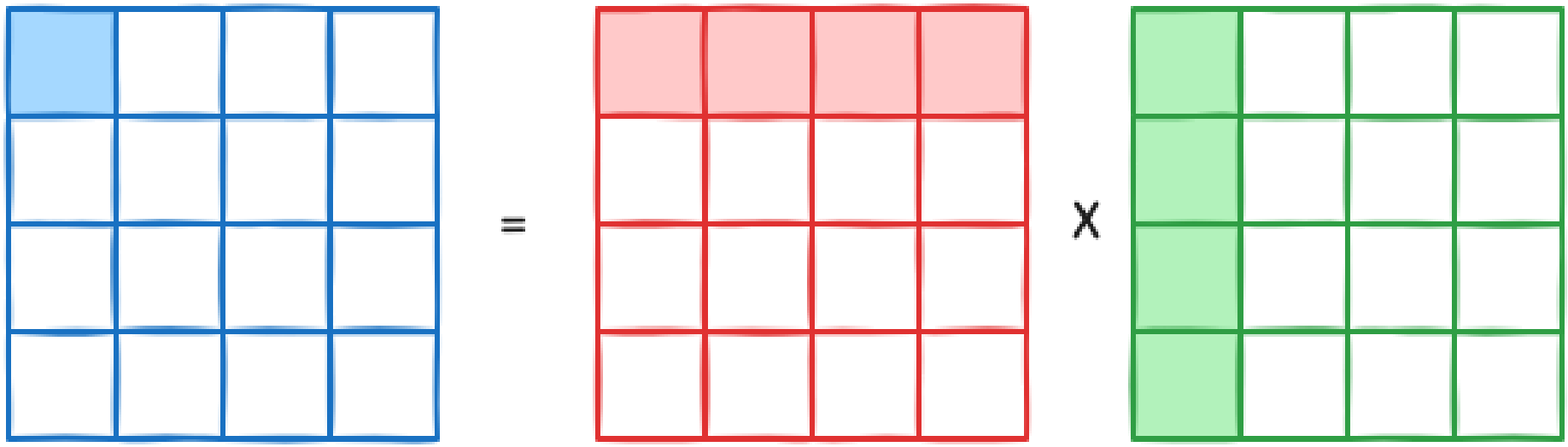


Each element of the output is independent. The results of C_{11} does not effect the results of C_{12}, or any other element.

Remark

If matrix multiplication is a **parallel** task, and GPU's were designed for the **parallel** task of rendering graphics, can we repurpose GPU's to do matrix multiplication?

Matrix Multiplication is Parallel



Each element of the output is independent. The results of C_{11} does not effect the results of C_{12}, or any other element.

Remark

If matrix multiplication is a **parallel** task, and GPU's were designed for the **parallel** task of rendering graphics, can we repurpose GPU's to do matrix multiplication? (Yes.)

What is GEMM?

Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

Our goal is unchanged math with less data movement and more useful work per thread, and sometimes we need to accumulate matrix multiplications.

Example

During the forward pass when training a model, we accumulate the gradients from the entire batch.

Takeaway

- CPUs can't efficiently perform matrix multiplication at scale.
- Graphics rendering is a parallel task, and so is matrix multiplication. Thus, GPUs can be good at matrix multiplication.
- GEMM is a generalization of of matrix multiplication, and is the operation we will be discussing.

Whats next

We have established that GPUs *can* do matrix multiplication, but how do we actually get there?

Table of Contents

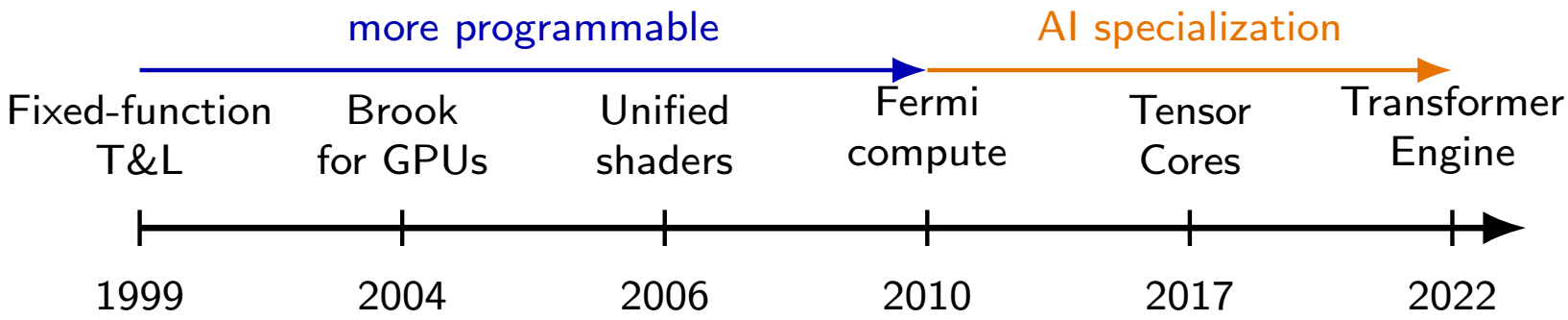
1. Neural Networks and the Matrix Multiplication
2. Why GPUs? What's wrong with CPUs?
3. From graphics pipeline to AI accelerator
4. Optimizing **Ge**neral **M**atrix **M**ultiplication (GEMM) on CUDA
5. Future discussions
6. Conclusion and References

Timeline

- **1999—2003:** fixed-function graphics gives way to programmable vertex and pixel shaders.
- **2004:** Brook for GPUs demonstrates a high-level stream-programming model on top of graphics APIs.
- **2006—2007:** GeForce 8800 introduces a unified shader architecture; NVIDIA releases CUDA publicly in 2007.
- **2010:** Fermi adds a coherent cache hierarchy, ECC support, and much stronger support for C++ and scientific computing.
- **2017:** Volta introduces Tensor Cores for mixed-precision matrix multiply–accumulate.
- **2020—2024:** Ampere broadens Tensor Core formats; Hopper adds the FP8 Transformer Engine; Blackwell adds further low-precision and AI-specific hardware.

What shaped modern GPU architecture?

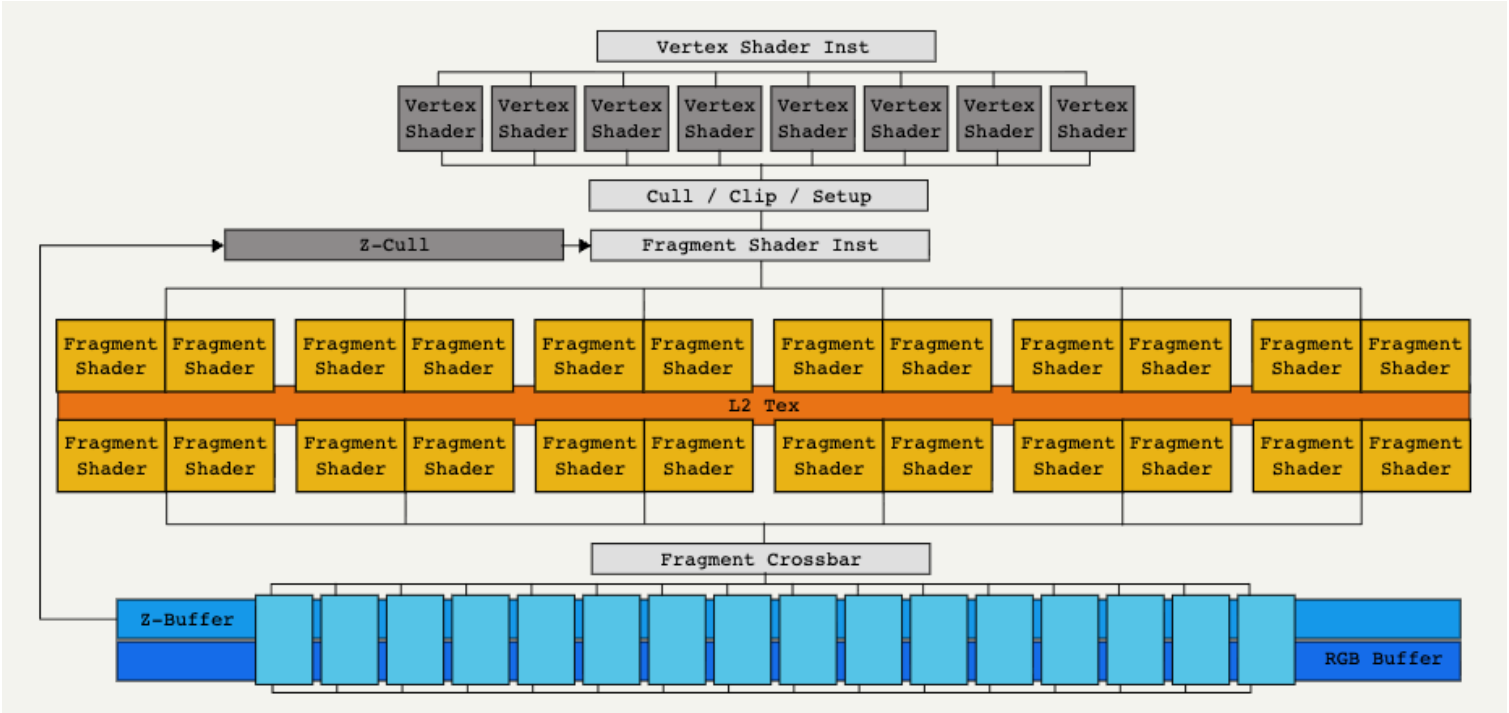
New workloads cause generalization/programmability, established workloads cause specialization.



What shaped modern GPU architecture?

<2006: graphics first

- GeForce 256 (1999) accelerated fixed-function transform and lighting.
- Later chips added programmable vertex and pixel shaders, but kept distinct stages and resources.

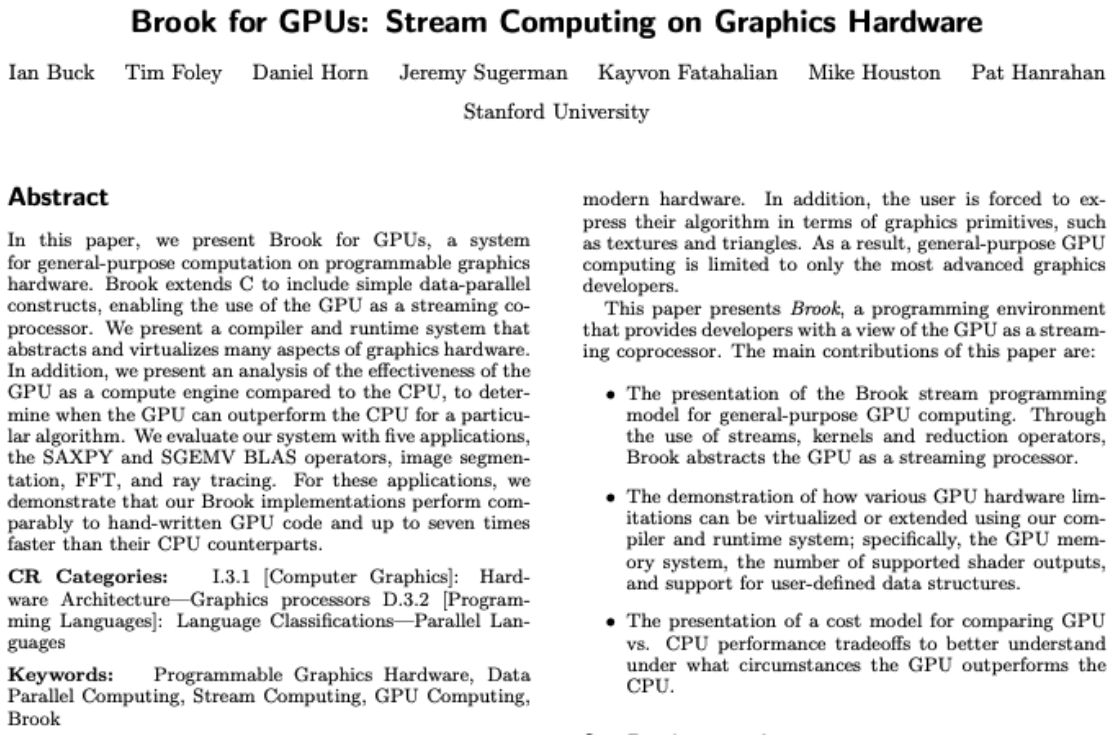


The G71 architecture (2005). Diagram source: Sanglard (2020).

What shaped modern GPU architecture?

2004: GPGPU before CUDA. Demand was always there.

- Brook for GPUs compiled a stream language to graphics APIs.
- It showed useful non-graphics workloads could exploit GPU throughput, despite an awkward programming model.



Brook for GPUs: Stream Computing on Graphics Hardware

Ian Buck Tim Foley Daniel Horn Jeremy Sugerman Kayvon Fatahalian Mike Houston Pat Hanrahan

Stanford University

Abstract

In this paper, we present Brook for GPUs, a system for general-purpose computation on programmable graphics hardware. Brook extends C to include simple data-parallel constructs, enabling the use of the GPU as a streaming co-processor. We present a compiler and runtime system that abstracts and virtualizes many aspects of graphics hardware. In addition, we present an analysis of the effectiveness of the GPU as a compute engine compared to the CPU, to determine when the GPU can outperform the CPU for a particular algorithm. We evaluate our system with five applications, the SAXPY and SGEMV BLAS operators, image segmentation, FFT, and ray tracing. For these applications, we demonstrate that our Brook implementations perform comparably to hand-written GPU code and up to seven times faster than their CPU counterparts.

CR Categories: I.3.1 [Computer Graphics]: Hardware Architecture—Graphics processors D.3.2 [Programming Languages]: Language Classifications—Parallel Languages

Keywords: Programmable Graphics Hardware, Data Parallel Computing, Stream Computing, GPU Computing, Brook

modern hardware. In addition, the user is forced to express their algorithm in terms of graphics primitives, such as textures and triangles. As a result, general-purpose GPU computing is limited to only the most advanced graphics developers.

This paper presents *Brook*, a programming environment that provides developers with a view of the GPU as a streaming coprocessor. The main contributions of this paper are:

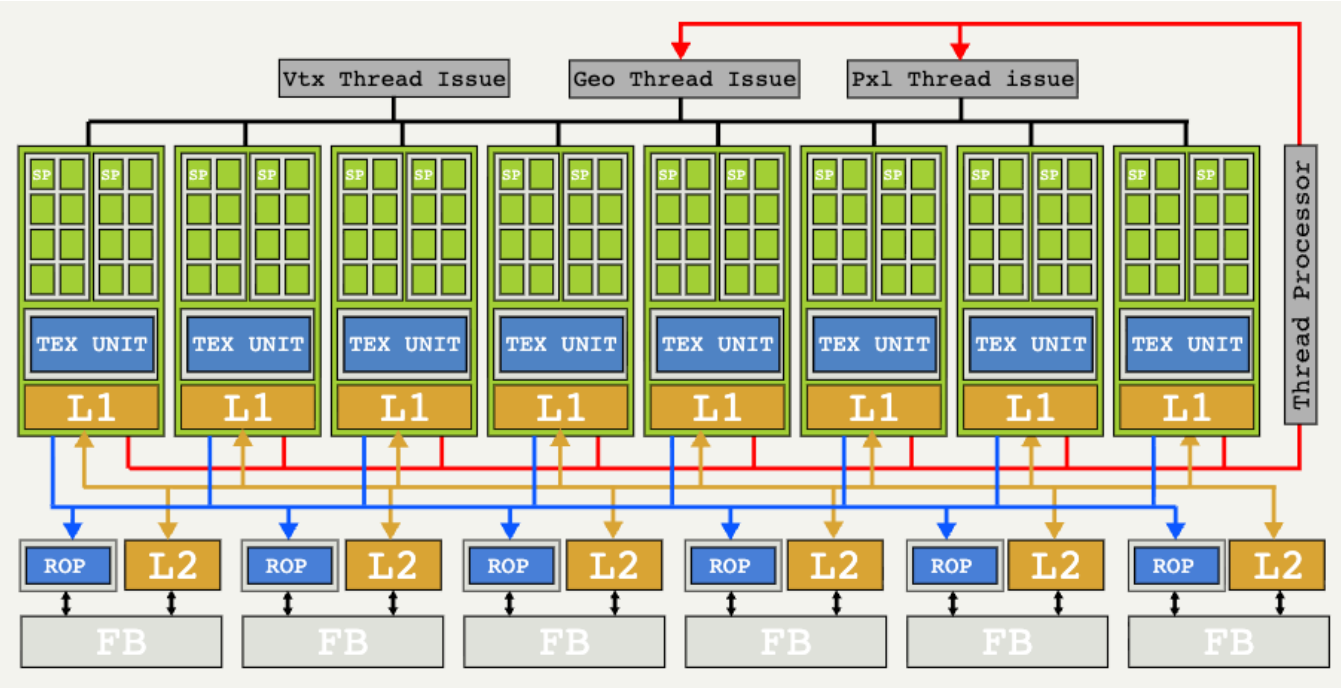
- The presentation of the Brook stream programming model for general-purpose GPU computing. Through the use of streams, kernels and reduction operators, Brook abstracts the GPU as a streaming processor.
- The demonstration of how various GPU hardware limitations can be virtualized or extended using our compiler and runtime system; specifically, the GPU memory system, the number of supported shader outputs, and support for user-defined data structures.
- The presentation of a cost model for comparing GPU vs. CPU performance tradeoffs to better understand under what circumstances the GPU outperforms the CPU.

Buck et al., "Brook for GPUs," *ACM TOG* 23(3), 2004.

What shaped modern GPU architecture?

2006-2007: increasing complexity necessitates unified compute

- G80 ran vertex, geometry, and pixel work on unified pool of **Streaming Multiprocessors** (SM's).
- Publicly released in 2007, CUDA exposed this parallel machine without graphics API tricks.

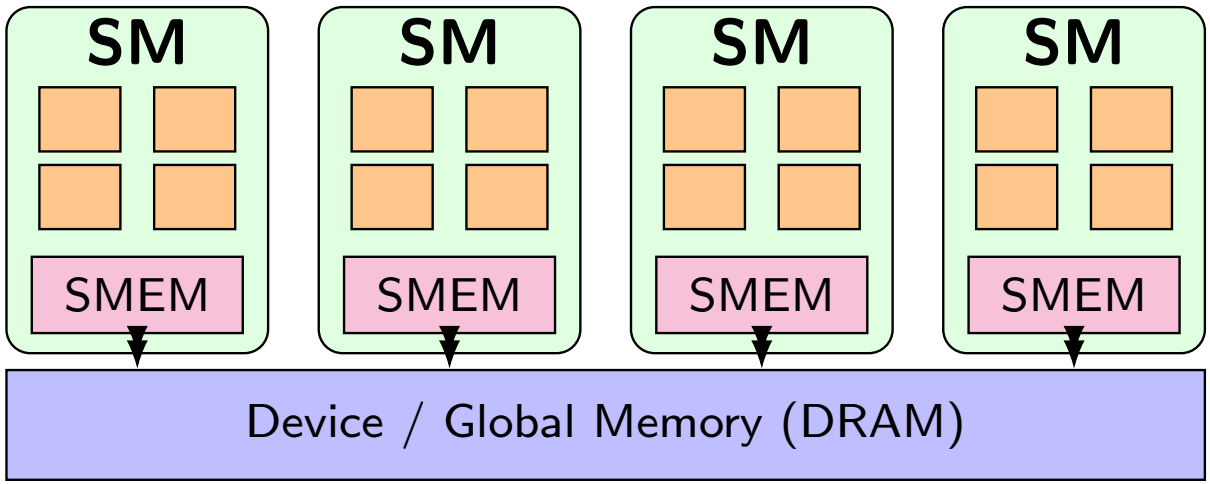


G80 block diagram. Diagram source: Sanglard (2020); architecture described by Lindholm et al. (2008).

Takeaways

Our basic model of the GPU:

- SMs: primary compute unit
- CUDA Cores: like the "ALUs" of the SMs.
- L1 cache/Shared memory: cache for the SM
- Device/Global Memory: like RAM for a normal CPU system.



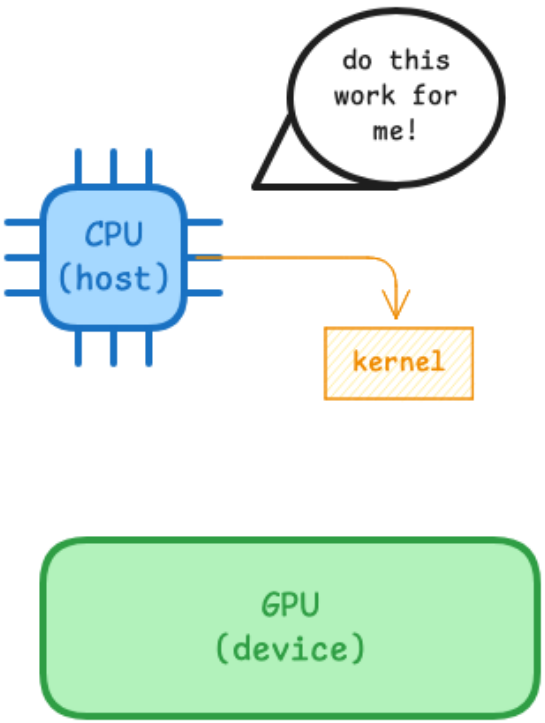
Whats next

How can we actually implement a matrix multiplication on a GPU? What tools and mental models do we need?

Table of Contents

1. Neural Networks and the Matrix Multiplication
2. Why GPUs? What's wrong with CPUs?
3. From graphics pipeline to AI accelerator
4. Optimizing **Ge**neral **M**atrix **M**ultiplication (GEMM) on CUDA
5. Future discussions
6. Conclusion and References

CUDA Basics




```
int main() {
  const int M = 128, N = 128, K = 128;

  float *dA, *dB, *dC;
  cudaMalloc(&dA, M * K * sizeof(float);
  cudaMalloc(&dB, K * N * sizeof(float));
  cudaMalloc(&dC, M * N * sizeof(float));

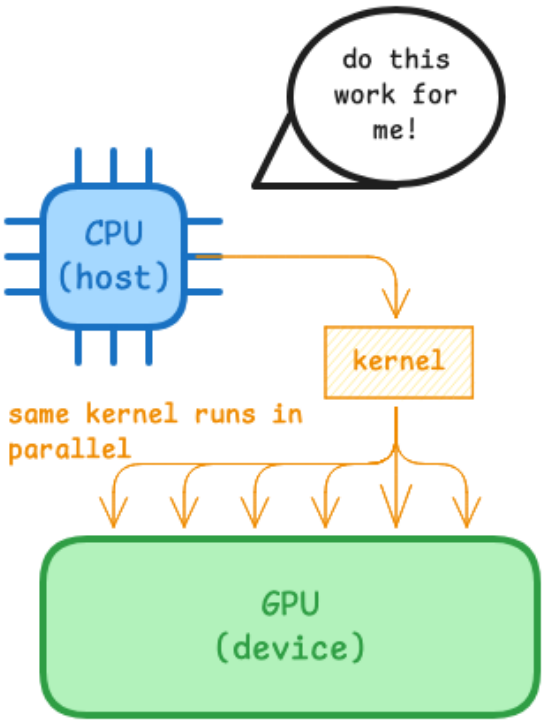
  // load values into these device pointers...

  run_sgemm_naive(M, N, K, 1, dA, dB, 0, dC);

  cudaFree(dA); cudaFree(dB); cudaFree(dC);
}
```

- The CPU is known as the **host**, and manages top level logic.
- The GPU is known as the **device**, and does the heavy lifting.
- The host launches **kernels** to the device.

CUDA Basics

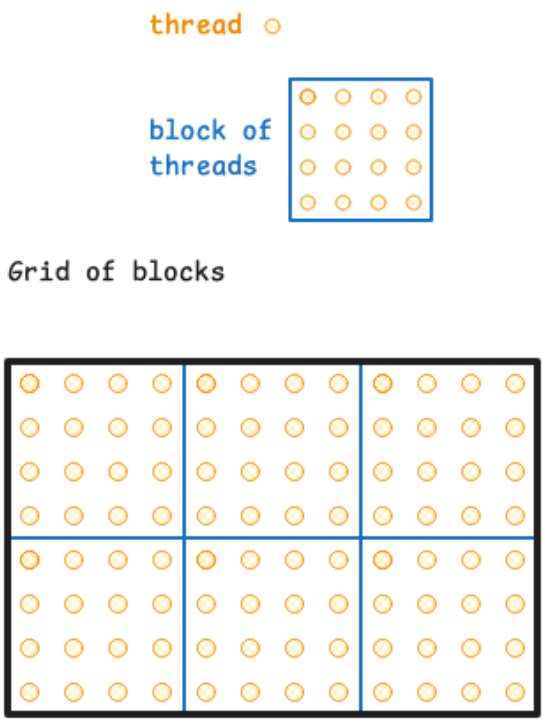



```
__global__ void sgemm_naive(
  int M, int N, int K, float alpha, const float *A,
  const float *B, float beta, float *C
) {
  // implement kernel ...
}

void run_sgemm_naive(int M, int N, int K, float alpha,
     float *A, float *B,
                       float beta, float *C) {
  dim3 blockDim(32, 32);
  dim3 gridDim(CEIL_DIV(M, 32), CEIL_DIV(N, 32));
  sgemm_naive<<<gridDim, blockDim>>>(
      M, N, K, alpha, A, B, beta, C);
}
```

- A kernel runs in parallel on many **threads** on the device.
- The number of threads is configured by the host, when it **launches** the kernel.

CUDA Basics




```
__global__ void sgemm_naive(
  int M, int N, int K, float alpha, const float *A,
  const float *B, float beta, float *C
) {
  // implement kernel ...
}

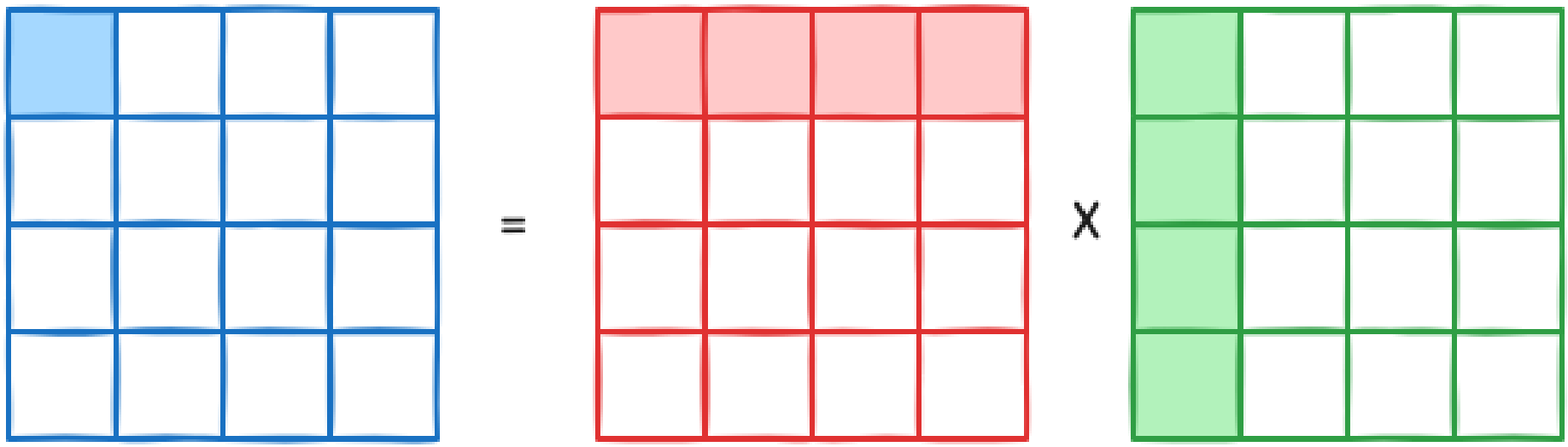
void run_sgemm_naive(int M, int N, int K, float alpha,
     float *A, float *B,
                       float beta, float *C) {
  dim3 blockDim(32, 32);
  dim3 gridDim(CEIL_DIV(M, 32), CEIL_DIV(N, 32));
  sgemm_naive<<<gridDim, blockDim>>>(
      M, N, K, alpha, A, B, beta, C);
}
```

- Threads are grouped into **blocks**.
- The collection of blocks requested by the host is organized in a **grid**.

One thread per output element

Recall

Recall that matrix multiplication is naturally parallel, and the output elements of C can be computed independently in parallel.

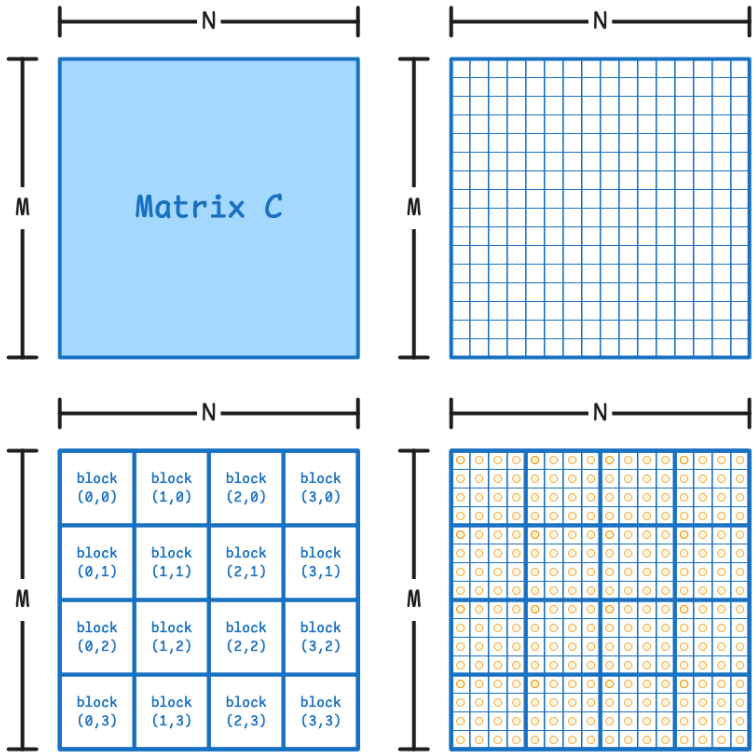


We can naturally map one thread to each output element!

Naive SGEMM Kernel

```
__global__ void sgemm_naive(
    int M, int N, int K, float alpha, const float *A,
    const float *B, float beta, float *C
) {
  const uint x = blockIdx.x * blockDim.x + threadIdx.x;
  const uint y = blockIdx.y * blockDim.y + threadIdx.y;

  if (x < M && y < N) {
    float tmp = 0.0;
    for (int i = 0; i < K; ++i)
      tmp += A[x*K+i] * B[i*N+y];
    C[x*N+y] = alpha*tmp + beta*C[x*N+y];
  }
}
void run_sgemm_naive(
  int M, int N, int K, float alpha, float *A,
  float *B, float beta, float *C
) {
  dim3 blockDim(32, 32);
  dim3 gridDim(CEIL_DIV(M, 32), CEIL_DIV(N, 32));
  sgemm_naive<<<gridDim, blockDim>>>(
      M, N, K, alpha, A, B, beta, C);
}
```



In Context

```
__global__ void sgemm_naive(int M, int N, int K, float alpha, const float *A,
                            const float *B, float beta, float *C) {
  const uint x = blockIdx.x * blockDim.x + threadIdx.x;
  const uint y = blockIdx.y * blockDim.y + threadIdx.y;

  if (x < M && y < N) {
    float tmp = 0.0;
    for (int i = 0; i < K; ++i)
      tmp += A[x*K+i] * B[i*N+y];
    C[x*N+y] = alpha*tmp + beta*C[x*N+y];
  }
}

void run_sgemm_naive(int M, int N, int K, float alpha, float *A, float *B,
                     float beta, float *C) {
  dim3 blockDim(32, 32);
  dim3 gridDim(CEIL_DIV(M, 32), CEIL_DIV(N, 32));
  sgemm_naive<<<gridDim, blockDim>>>(
      M, N, K, alpha, A, B, beta, C);
}

int main() {
  // ... set up matrices and dimensions ...
  run_sgemm_naive(M, N, K, alpha, A, B, beta, C);
}
```

Device
`__global__` marks a GPU **kernel**.

Many threads
Every thread runs this code with its own ID.

Host
CPU code manages the top-level logic.

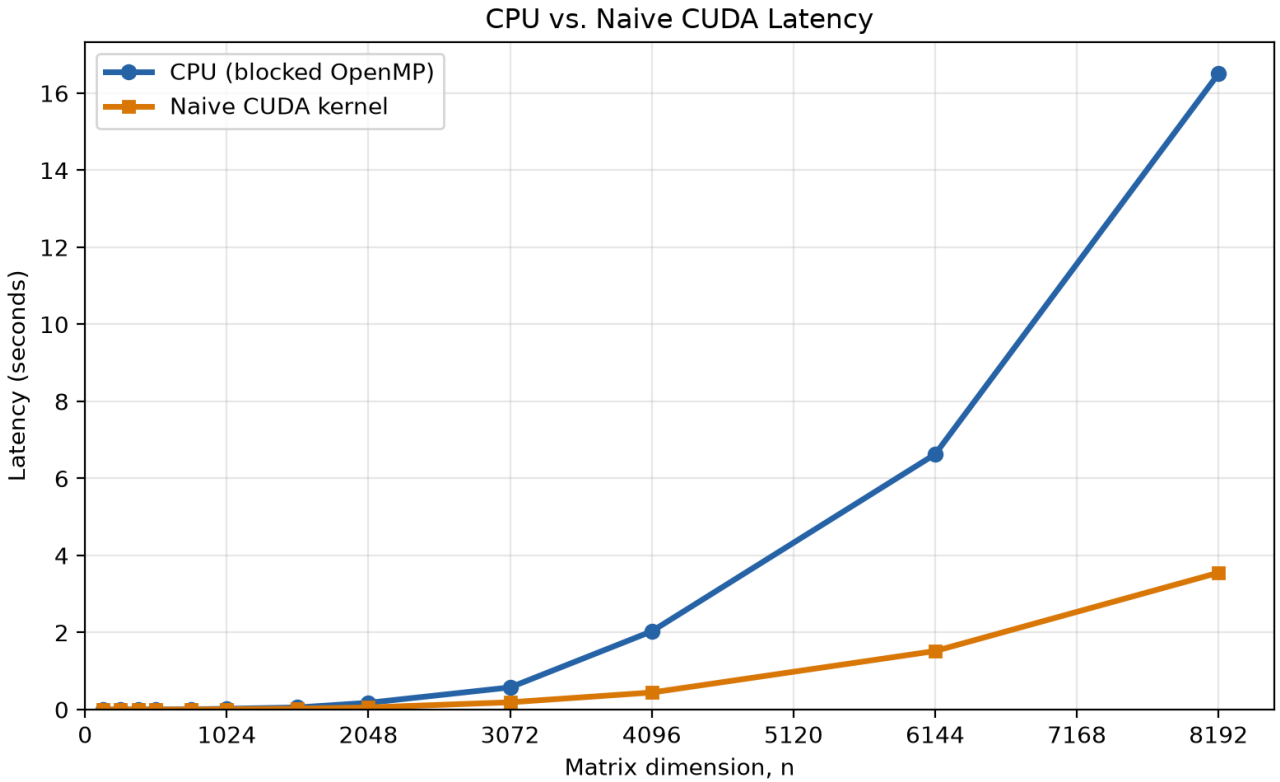
Launch
The host chooses the grid and block sizes.

Naive SGEMM visualization

- One thread computes one element of C.
- It walks one row of A and one column of B for the full dot product.
- Neighboring threads repeatedly fetch many of the same values from global memory.

Play naive-kernel visualization

Naive CUDA is already faster than the CPU



We can do better

These benchmarks were run on an RTX 3090 with specs:

- 10,496 CUDA cores over 82 SM's with up to 35.6 TFLOP/s of FP32 throughput.
- 24 GB of GDDR6X memory with 936 GB/s of bandwidth.

Theoretical lower bound on latency

For SGEMM with $n = 8192$:

$$\text{work} \approx 2n^3 = 1.10 \times 10^{12} \text{ FLOP},$$

$$t_{\min} \approx \frac{1.10 \text{ TFLOP}}{35.6 \text{ TFLOP/s}} \approx 0.031 \text{ s} = 31 \text{ ms}.$$

Observed latency: 3.55 **s—about** 115× **the theoretical minimum.**

What can we take advantage of?

1. **Accessing Device Memory is very slow.** 96.7% of warp stalls are memory-related.
2. Blocks are scheduled to SMs; threads are scheduled on CUDA cores within SMs.
3. Threads can save time by loading data from Device/Global memory into Shared Memory within the SM.

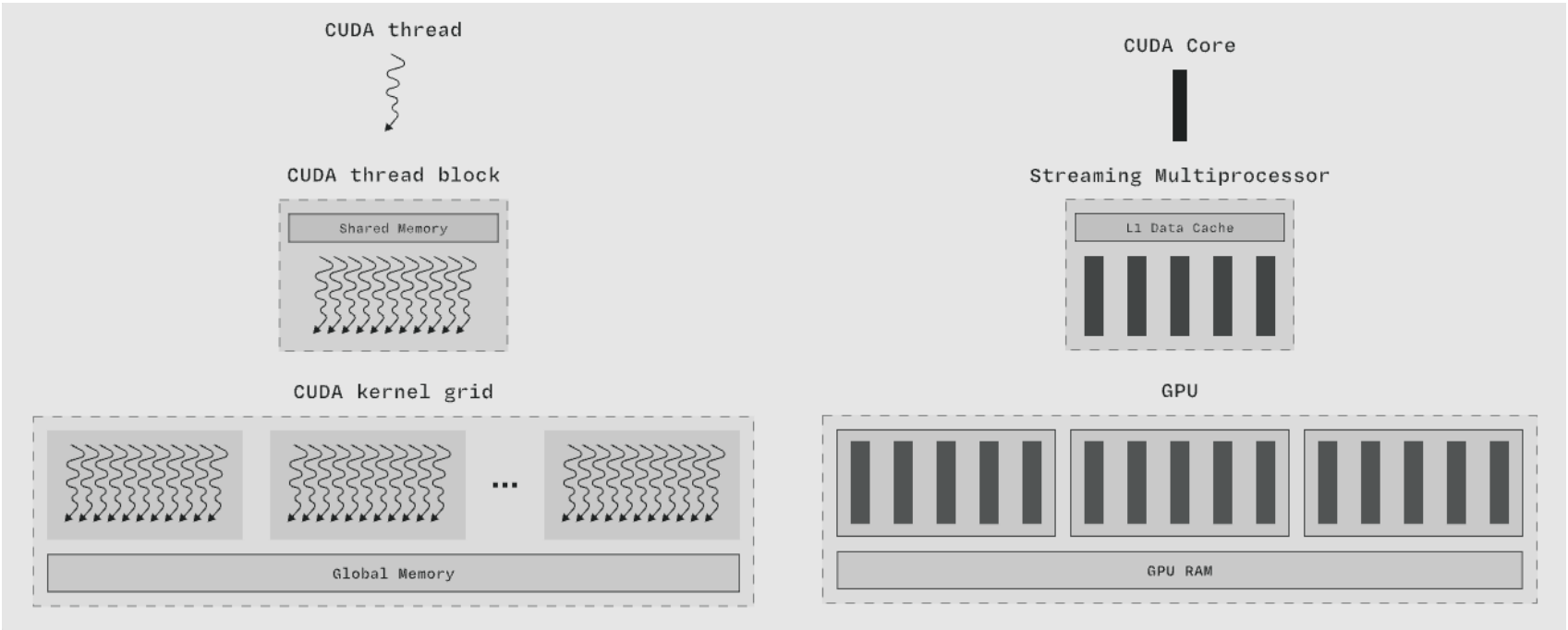
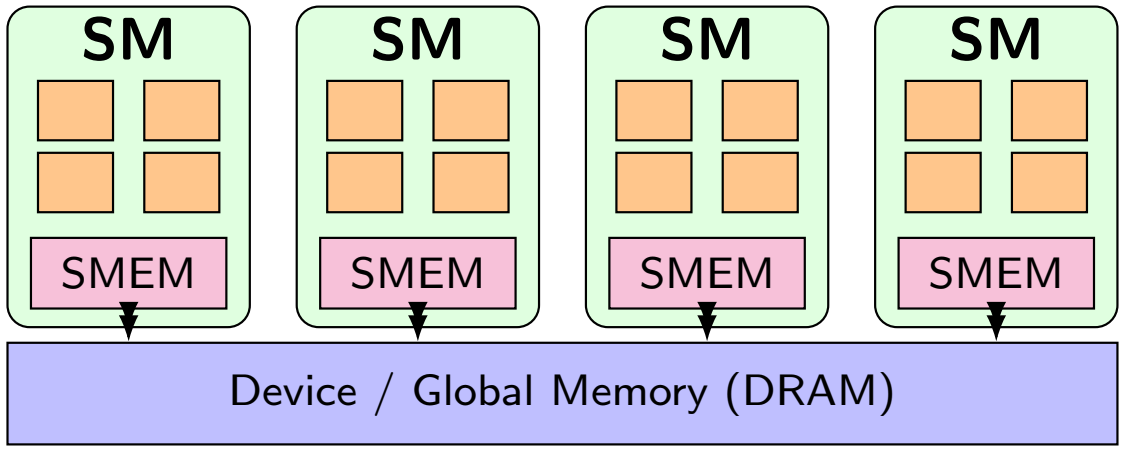


Image source: Modal, "CUDA Programming Model," *GPU Glossary*.

Tiling

Main Idea

Before doing any computation, load a chunk of the necessary data from DRAM to SMEM as a block. Then, memory accesses for the computation now only go to SMEM.



Basic tiled SGEMM: reuse data on chip

1. A block cooperatively loads a tile of A and a tile of B from global memory into shared memory.
2. Synchronize so every thread sees the complete tiles.
3. Each thread accumulates a partial dot product using the cached values.
4. Synchronize, advance along K, and repeat until the output tile is complete.

Basic tiled SGEMM

```
__shared__ float As[BLOCKSIZE * BLOCKSIZE];
__shared__ float Bs[BLOCKSIZE * BLOCKSIZE];
float tmp = 0.0;

for (int bkIdx = 0; bkIdx < K; bkIdx += BLOCKSIZE) {
  As[threadRow * BLOCKSIZE + threadCol] =
      A[threadRow * K + threadCol];
  Bs[threadRow * BLOCKSIZE + threadCol] =
      B[threadRow * N + threadCol];

  __syncthreads();
  A += BLOCKSIZE;
  B += BLOCKSIZE * N;

  for (int dotIdx = 0; dotIdx < BLOCKSIZE; ++dotIdx) {
    tmp += As[threadRow * BLOCKSIZE + dotIdx] *
           Bs[dotIdx * BLOCKSIZE + threadCol];
  }

  __syncthreads();
}
```

Cooperate
Each thread loads one value from A and one from B.

Synchronize
Wait until both shared tiles are complete.

Reuse
One thread accumulates from the cached row and column.

Advance
Protect the cache, move along K, and repeat.

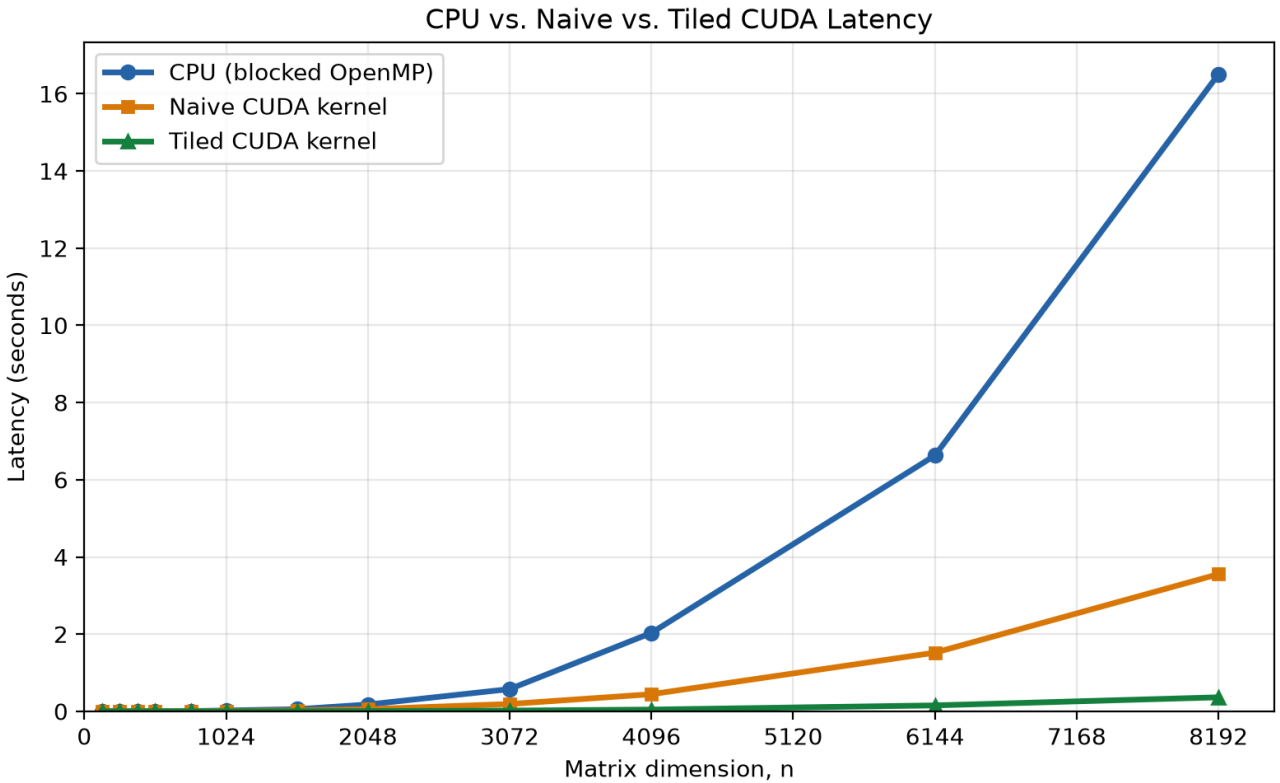
Adapted from Boehm (2022), kernel 3.

Basic tiled SGEMM visualization

- Watch each block load small regions of A and B together.
- The loaded tiles are reused to update an entire tile of C.
- Synchronization separates loading, computing, and advancing to the next tiles.

Play tiled-kernel visualization

Tiling cuts latency by another order of magnitude



Can we still do better?

Throughput analysis

We report a final latency of 0.36s or 360ms for $n = 8192$. This is around $10\times$ the theoretical minimum of 31ms.

After profiling, we see that 76.4% of warp stalls are still memory-related. In other words, we are **memory bound**.

Memory bound means poor hardware utilization.

If a kernel is **memory bound** we can increase throughput by moving less data from DRAM. Since we have to read the entirety of both input matrices at least once in order to perform the computation, this must be done through **data reuse** via caching in SMEM. In general, we can't decrease the total FLOPs required for an operation.

Arithmetic intensity and the Roofline Model

Throughput

Formally stated, **throughput** is the total useful work (FLOPs) done per unit time. *We previously discussed theoretical peak throughput*.

$$\text{Throughput} = \frac{\text{FLOPs}}{\text{time}}$$

Example

Theoretical peak throughput of our 3090 is 35.6 TFLOP/s. We observed 360ms or 0.36s of latency for $n = 8192$, where the total FLOPs required is 1.10 TFLOPs.

Thus, we have an observed throughput of

$$\frac{1.10\text{TFLOPs}}{0.36\text{s}} \approx 3.05\text{TFLOP/s}.$$

Roughly 10% of peak throughput.

Arithmetic intensity and the Roofline Model

Arithmetic Intensity

We define the **Arithmetic Intensity** (AI) of our kernel as the number of FLOPs done per bytes moved from DRAM.

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}$$

Example

For $n = 8192$ and 32×32 tiles, the grid contains $256^2 = 65{,}536$ blocks. Each block takes 256 steps, loading one tile from both A and B per step, then writes one tile to C.

$$\text{bytes/block} = \left(256 \times 2 \times 32^2 + 32^2\right) \times 4 = 2{,}101{,}248 \text{ bytes},$$

$$\text{bytes total} = 65{,}536 \times 2{,}101{,}248 \approx \boxed{137.7 \text{ GB}}.$$

Arithmetic intensity and the Roofline Model

Example (cont.)

For $n = 8192$ and 32×32 tiles, the grid contains $256^2 = 65{,}536$ blocks. Each block takes 256 steps, loading one tile from both A and B per step, then writes one tile to C. Across the complete grid, the kernel moves

$$65{,}536 \left(256 \times 2 \times 32^2 + 32^2\right) \times 4 \approx 137.7 \text{ GB}.$$

With a total work count of 1.10 TFLOP, its arithmetic intensity is

$$\text{AI} = \frac{1.10 \times 10^{12} \text{ FLOP}}{137.7 \times 10^9 \text{ bytes}} \approx \boxed{8.0 \text{ FLOP/byte}}.$$

Arithmetic intensity and the Roofline Model

Roofline Model

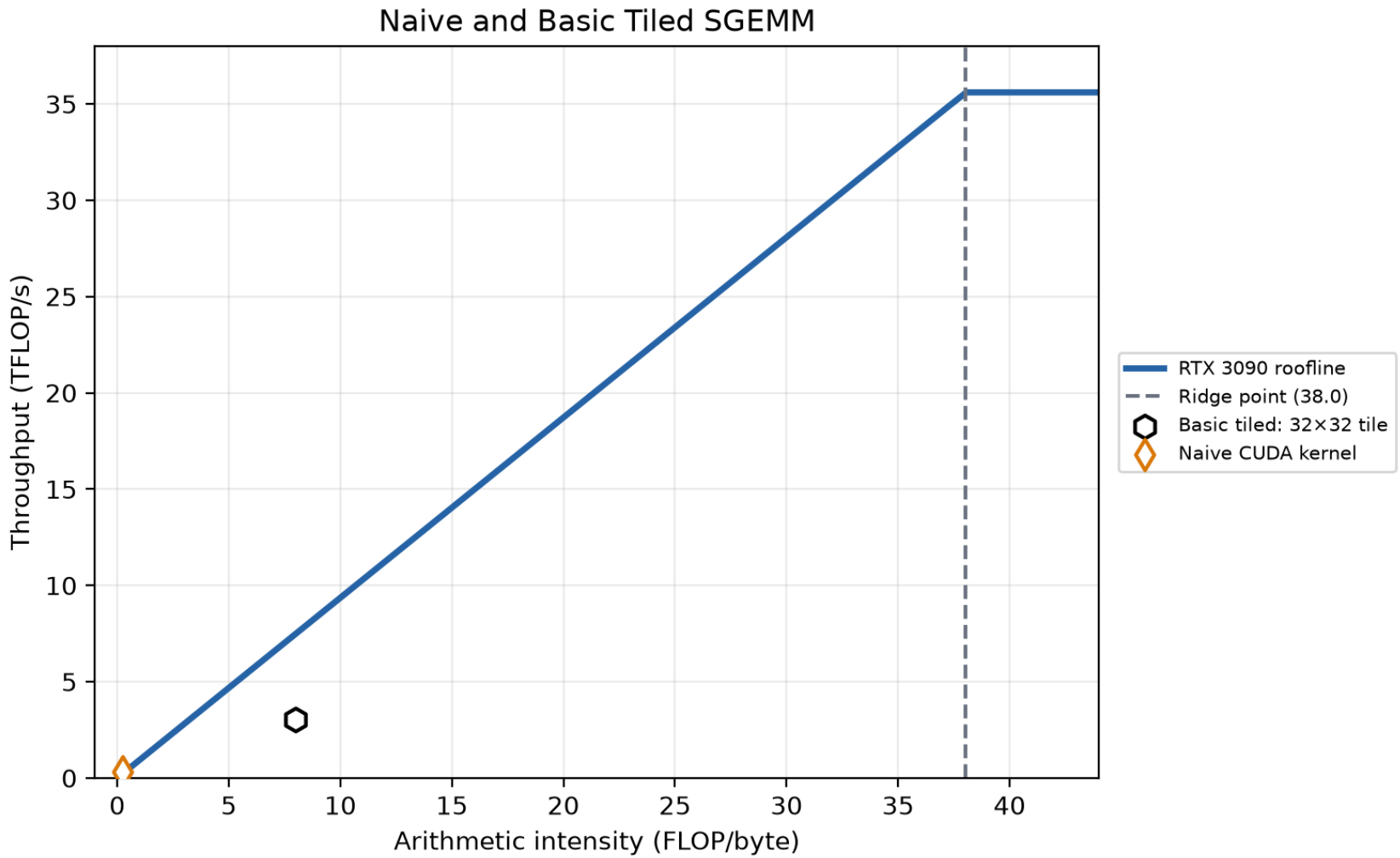
For a given system, we can predict a the upper bound of throughput of a kernel given its AI. Up to a certain point, increasing AI will raise the upper bound of throughput. However, at the **ridge point**, we become **compute bound**, and no longer benefit from increasing AI.

Example

For our 3090, we have 35.6 TFLOP/s of peak FP32 throughput and 936 GB/s of memory bandwidth. Thus, our ridge point is

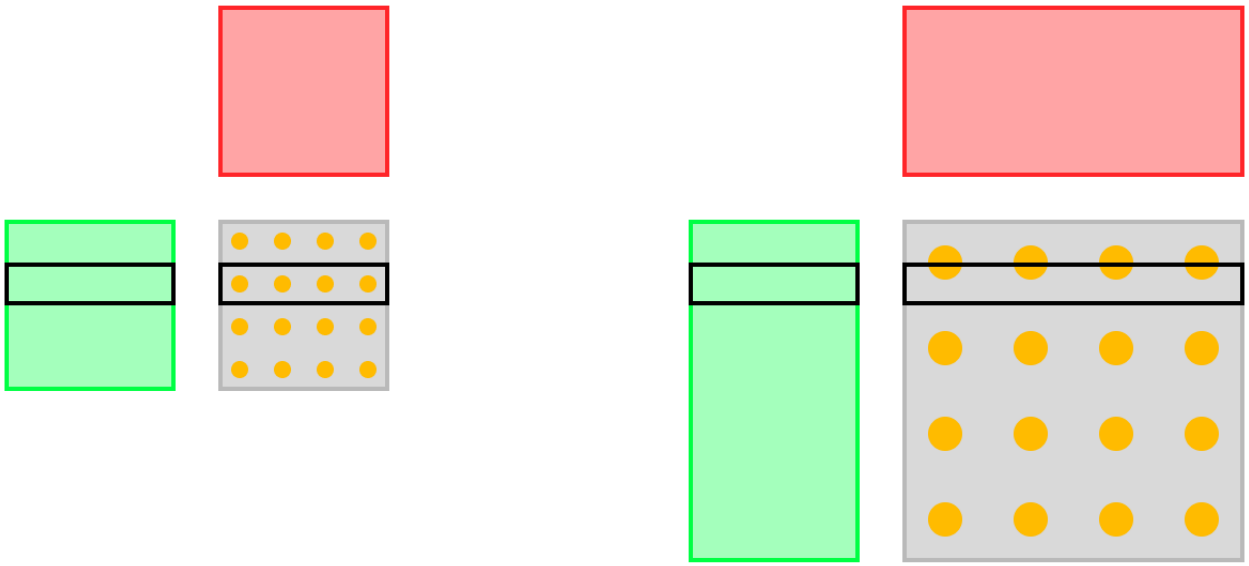
$$\text{RP} = \frac{35.6 \times 10^{12} \text{FLOP/s}}{936 \times 10^{9} \text{bytes/s}} \approx 38.0 \text{FLOPs/byte}$$

Arithmetic intensity and the Roofline Model



Thread coarsening: Increasing AI

- Basic tiling assigns 1 element of the output to each thread.
- We can *coarsen* each thread, and assign a $TM \times TN$ portion of the output to each thread.
- We load larger chunks into SMEM, increasing data reuse and AI.
- One loaded A value can update several columns; one loaded B value can update several rows.

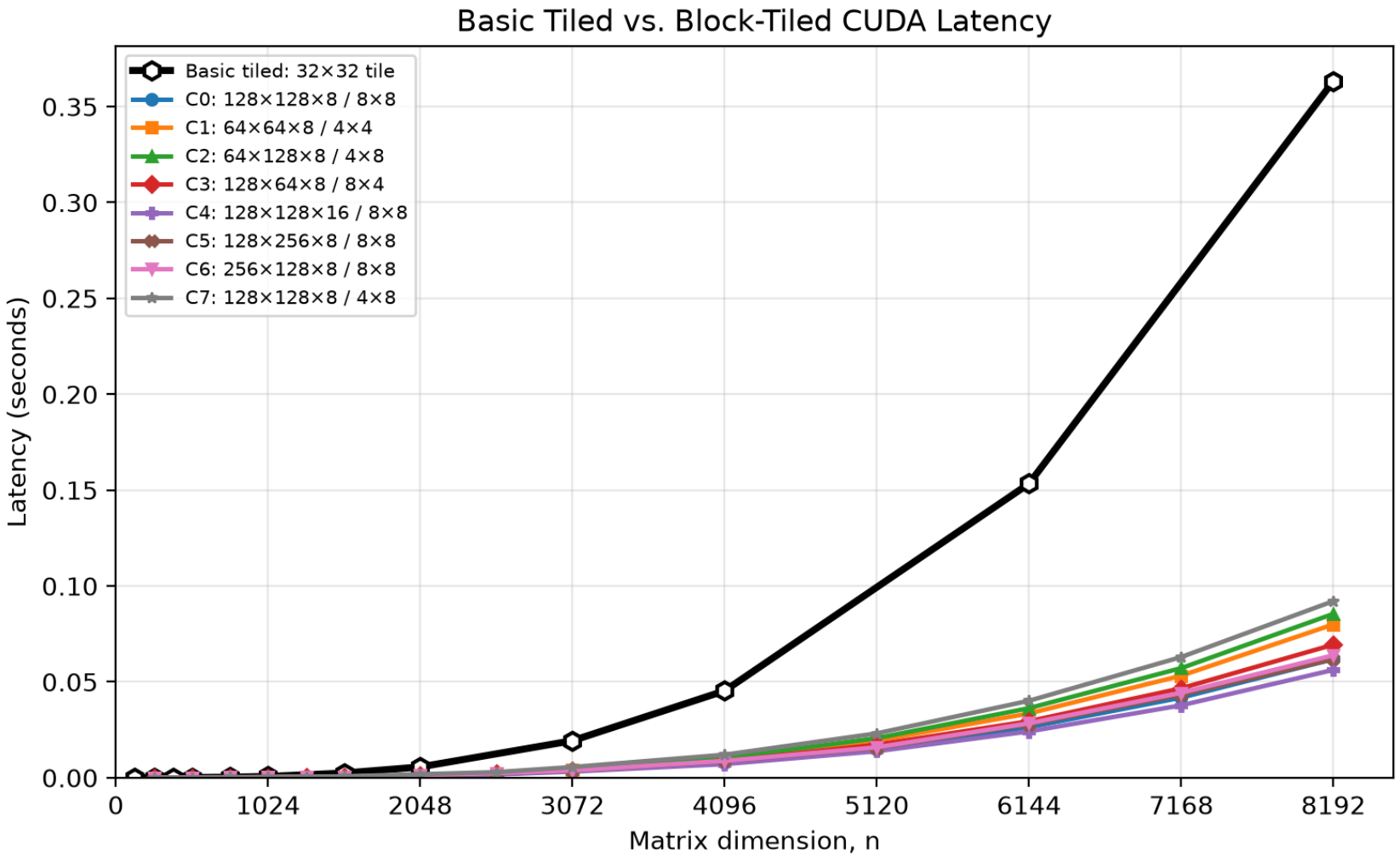


Coarsened SGEMM visualization

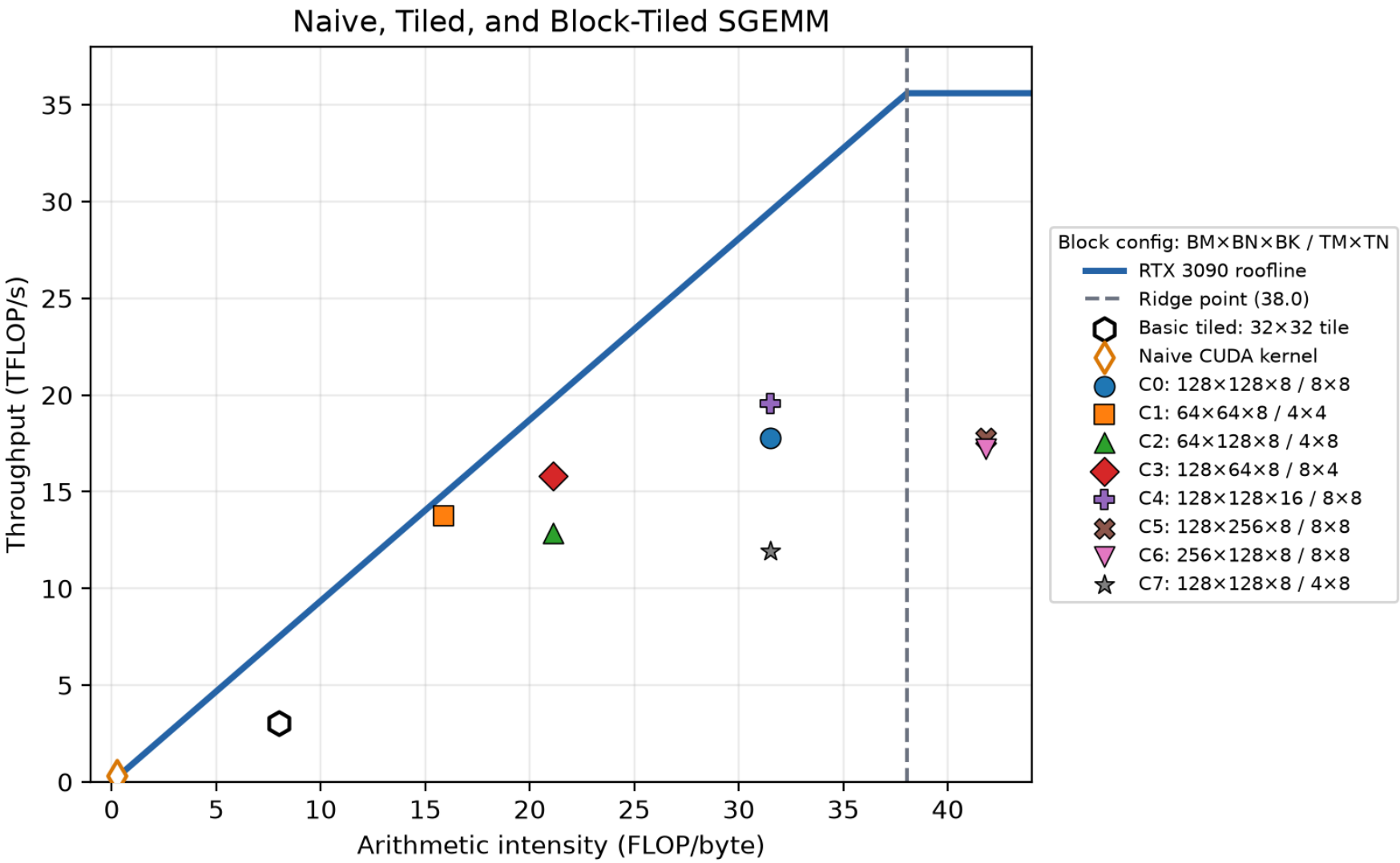
- Each thread now owns a small two-dimensional region of C.
- Values from shared memory are cached in registers and reused across that region.
- More outputs per thread means fewer memory operations per result.

Play coarsened-kernel visualization

Basic tiled vs. block-tiled latency



Tiling configurations on the roofline



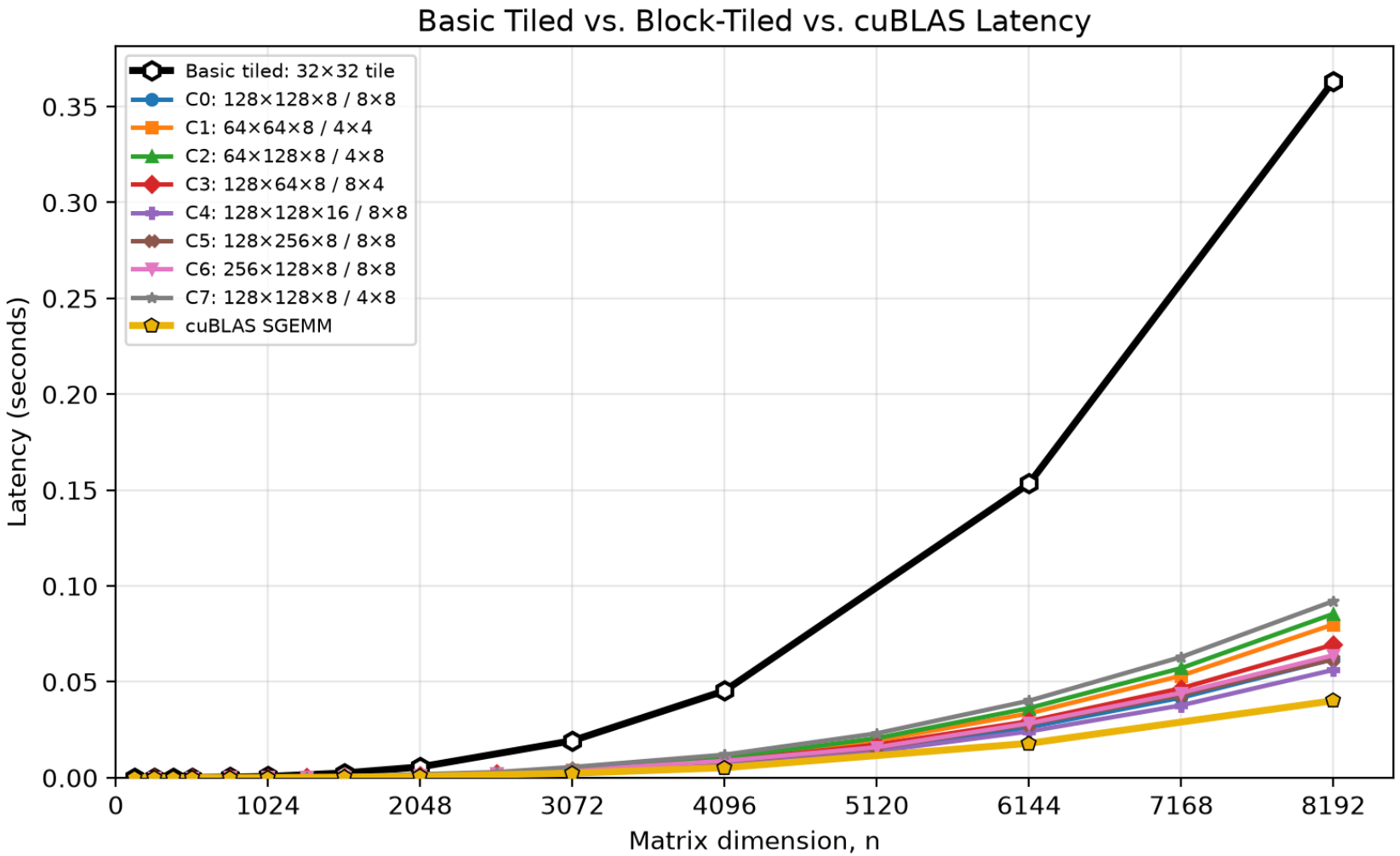
We have only scratched the surface: cuBLAS

- cuBLAS is NVIDIA's architecture-tuned linear algebra library and the practical performance target for our SGEMM kernels.
- At $n = 8192$, cuBLAS reaches 27.38 TFLOP/s, while our fastest configuration (C4) reaches 19.54 TFLOP/s, or 71.4% of cuBLAS's throughput.
- cuBLAS completes the multiplication in 40.2 ms versus 56.3 ms for C4, about 1.40× faster.

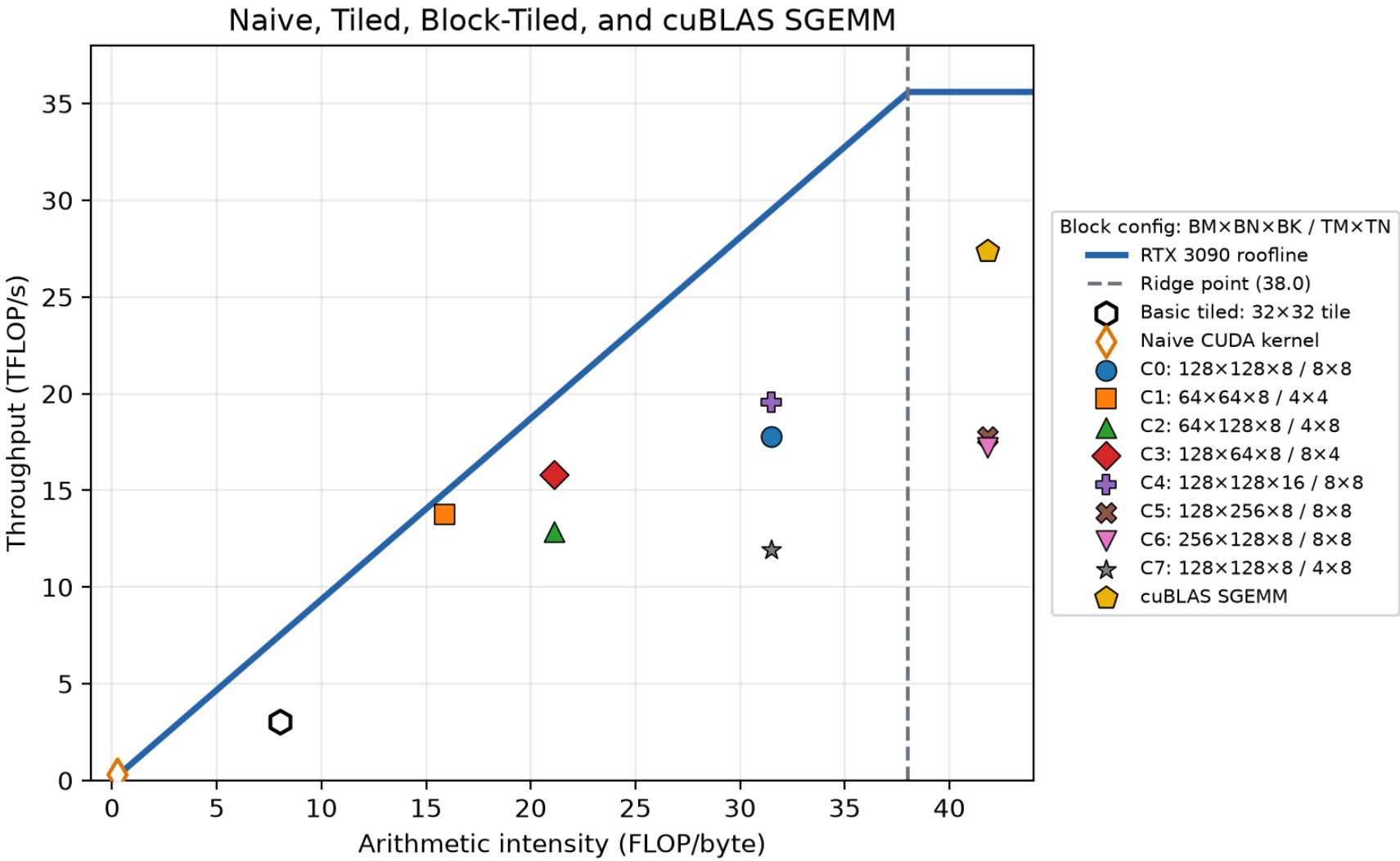
The remaining gap

Production kernels combine deeper register and warp tiling, vectorized memory access, pipelined loads, and architecture-specific instruction scheduling.

Basic tiled vs. block-tiled latency vs. cuBLAS



cuBLAS on the roofline



cuBLAS arithmetic intensity is estimated from the profiled 256×128 output-tile shape.

We've come a long way

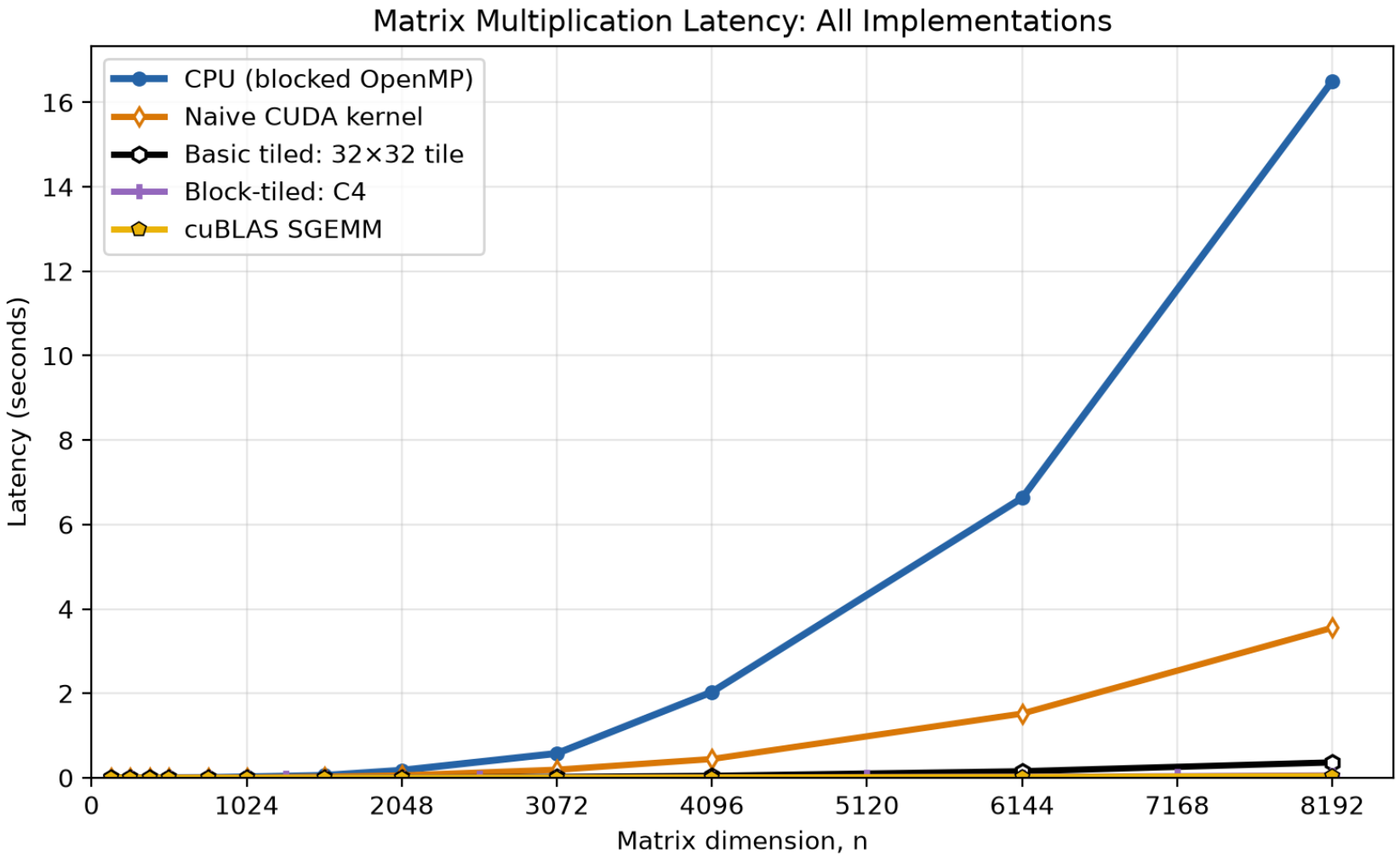


Table of Contents

1. Neural Networks and the Matrix Multiplication
2. Why GPUs? What's wrong with CPUs?
3. From graphics pipeline to AI accelerator
4. Optimizing **Ge**neral **M**atrix **M**ultiplication (GEMM) on CUDA
5. Future discussions
6. Conclusion and References

Specialized kernels

A general SGEMM kernel must handle many shapes and layouts. Real models let us trade some of that flexibility for more performance.

What can be specialized?

- Fixed tensor shapes and memory layouts
- Reduced precision and Tensor Core tile sizes
- Structured sparsity or quantized weights
- Fused bias, activation, and normalization

Why does it help?

- Less indexing and control-flow overhead
- Fewer kernel launches and intermediate writes
- Better use of registers and specialized hardware
- Autotuning can choose a kernel for each layer

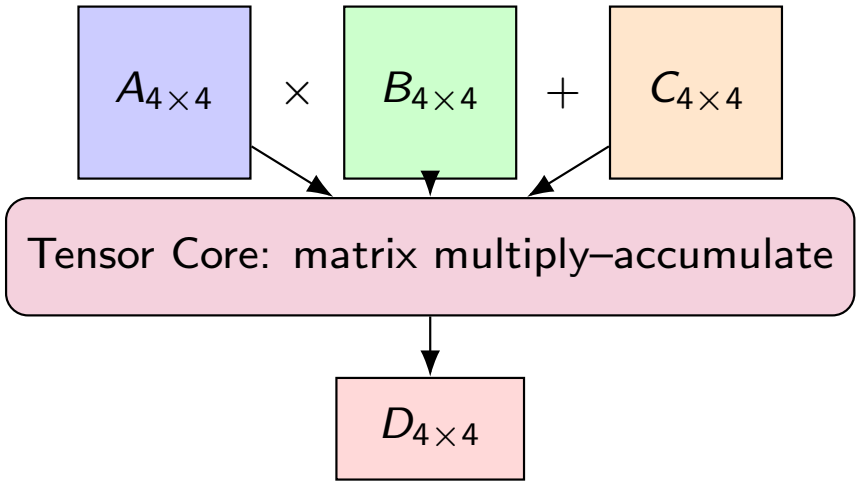
Example: fuse the epilogue

Compute $Y = \mathrm{ReLU}(XW + b)$ in one kernel instead of writing XW to global memory and reading it back for bias and activation.

Continuing GPU evolution

2017: Volta Tensor Cores

- Tensor Cores perform mixed-precision matrix multiply–accumulate: $D = AB + C$.
- The original Volta unit multiplies FP16 inputs and accumulates in FP16 or FP32.
- Matrix multiplication becomes an explicit hardware primitive, alongside general-purpose CUDA cores.



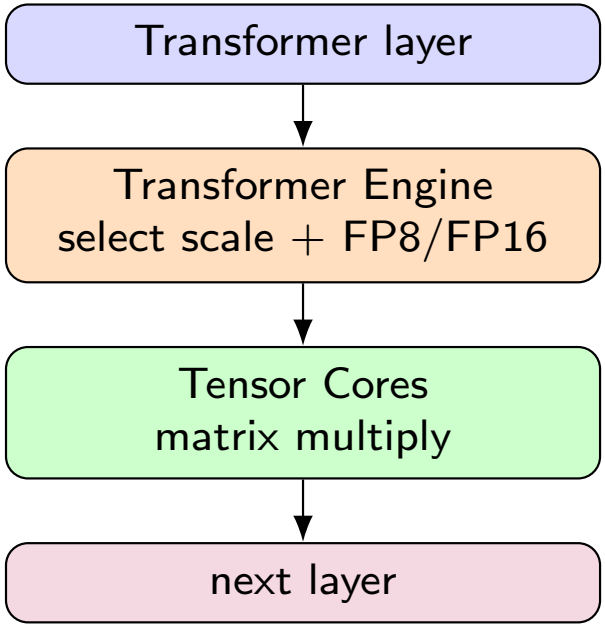
Operation shape shown for first-generation Volta Tensor Cores. Source:

NVIDIA, *Volta Architecture Whitepaper*, 2017.

Continuing GPU evolution

2020 onward: specialize around AI dataflow

- Ampere (2020) adds TF32 and structured sparsity support.
- Hopper (2022) adds FP8 Tensor Cores and a Transformer Engine that dynamically chooses FP8 or FP16 per layer.
- Blackwell (2024) adds more low-precision formats and a second-generation Transformer Engine. This is more than incremental scaling.



Source: NVIDIA, *Hopper Architecture In-Depth* (2022).

Continuing GPU evolution

2026: Vera Rubin—optimization moves to the rack

- The target is no longer an isolated GPU, but a complete rack-scale computing platform.
- Vera CPUs, Rubin GPUs, memory, NVLink, and networking are designed together to run one distributed AI workload.
- A high-bandwidth scale-up fabric lets many accelerators share model parameters and intermediate results.
- Performance now depends on communication, power, cooling, and software orchestration as well as matrix throughput.

Unit of optimization

Kernel

↓

GPU

↓

Server

↓

Rack-scale system

Source: NVIDIA, *Vera Rubin Architecture*, announced roadmap.

Alternative accelerator architectures

Different vendor, or different architecture?

AMD Instinct: another GPU ecosystem

AMD's datacenter GPUs use many programmable compute units, matrix instructions, and high-bandwidth memory. ROCm provides the software stack used to program them.

Strength: retains GPU flexibility while competing with CUDA.
Tradeoff: performance depends heavily on software and library support, not only the silicon.

Google TPU: systolic arrays

Data moves rhythmically through a two-dimensional grid of multiply–accumulate units. Values are reused as they pass between neighboring units instead of repeatedly returning to memory.

Strength: efficient, predictable dense tensor operations.
Tradeoff: less flexible for irregular workloads.

Sources: AMD, *Instinct Accelerators*; Jouppi et al. (2017).

Alternative accelerator architectures

Other designs change the scale or location of computation.

Cerebras: wafer-scale integration

Many compute cores, distributed SRAM, and an on-wafer fabric are placed on one large processor. More communication stays on the wafer instead of crossing between accelerator packages.

Strength: memory capacity and bandwidth close to compute.
Tradeoff: difficult manufacturing, cooling, and mapping.

NPUs: AI inside the device

NPUs sit beside CPUs and GPUs in phones, laptops, and embedded systems. They accelerate supported quantized tensor operations under strict power and thermal limits.

Strength: private, low-latency inference without the cloud.
Tradeoff: limited memory and operator support.

Sources: Cerebras, *Wafer-Scale Engine*; Sze et al. (2017).

Alternative accelerator architectures

Model ASIC: Etched Sohu

The hardware and dataflow target transformer inference rather than arbitrary kernels. Removing general-purpose machinery can devote more silicon and energy to the intended model family.

Strength: potentially much higher throughput per watt.
Tradeoff: model changes can make fixed hardware obsolete.

The design spectrum

GPU: programmable

↓

TPU/NPU: operator-focused

↓

ASIC: model-focused

More specialization ⇒ less flexibility

There is no universally best architecture

Specialization can improve performance per watt, but reduces flexibility. The right balance depends on workload stability, scale, power, and the cost of moving data.

Source: Etched (2024), "Introducing Sohu."

Table of Contents

1. Neural Networks and the Matrix Multiplication
2. Why GPUs? What's wrong with CPUs?
3. From graphics pipeline to AI accelerator
4. Optimizing **Ge**neral **M**atrix **M**ultiplication (GEMM) on CUDA
5. Future discussions
6. Conclusion and References

Conclusion

1. **AI workloads expose parallel linear algebra.** Neural-network layers repeatedly reduce to matrix and tensor operations.
2. **GPUs already had the right foundation.** Graphics demanded parallel throughput and high memory bandwidth; unified shaders and CUDA made that hardware programmable.
3. **Data movement determines real performance.** Coalescing, shared-memory tiling, and register reuse move GEMM toward the compute limit.
4. **AI is now driving specialization.** Tensor Cores, lower precision, and new accelerators trade flexibility for throughput and efficiency.

Main takeaway

Graphics and AI both reward massive parallelism. The GPU succeeded by evolving from a graphics pipeline into a programmable matrix engine.

References: GPU evolution

- I. Buck et al. (2004), "Brook for GPUs: Stream Computing on Graphics Hardware."
- E. Lindholm et al. (2008), "NVIDIA Tesla: A Unified Graphics and Computing Architecture."
- NVIDIA (2009), *Fermi Compute Architecture Whitepaper*.
- NVIDIA (2017), *NVIDIA Tesla V100 GPU Architecture*.
- NVIDIA (2020), "NVIDIA Ampere Architecture In-Depth."
- NVIDIA (2022), "NVIDIA Hopper Architecture In-Depth."
- NVIDIA (2024), "NVIDIA Blackwell Architecture."
- NVIDIA, "NVIDIA Rubin Architecture."

References: CUDA and GEMM

- S. Boehm (2022), "How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance."
- S. Williams, A. Waterman, and D. Patterson (2009), "Roofline: An Insightful Visual Performance Model for Multicore Architectures."
- NVIDIA, *CUDA C++ Programming Guide*.
- NVIDIA, "GeForce RTX 3090 Family Specifications."
- NVIDIA, "CUDA Refresher: The CUDA Programming Model."
- NVIDIA, *cuBLAS Library Documentation*.
- Modal, "CUDA Programming Model," *GPU Glossary*.
- F. Sanglard (2020), "The CUDA Architecture."

References: alternative accelerators

- AMD, "AMD Instinct Accelerators."
- N. P. Jouppi et al. (2017), "In-Datacenter Performance Analysis of a Tensor Processing Unit."
- V. Sze et al. (2017), "Efficient Processing of Deep Neural Networks: A Tutorial and Survey."
- Cerebras, "The Wafer-Scale Engine."
- Etched (2024), "Introducing Sohu: The World's First Transformer ASIC."

Questions?